

Centro Universitário Internacional UNINTER
Análise e Desenvolvimento de Sistemas – EAD

Lúcius Aurélio Salgado Miranda

PROJETO E IMPLEMENTAÇÃO DE UMA API BACK-END PARA UMA REDE DE
LANCHONETES EM EXPANSÃO

Modelagem de Dados, API REST e Regras de Negócio

Brasília, DF
2026

Sumário

1. Estudo de Caso	2
2. Requisitos Funcionais.....	4
3. Requisitos Não Funcionais	6
4. Priorização das Funcionalidades	7
5. Regras de Negócio	8
6. Identificação dos atores e perfis de acesso.	10
7. Definição do fluxo crítico do sistema.	11
8. Diagrama de Entidade-Relacionamento	12
8. 1 Diagrama de casos de uso.	18
9. Diagrama de Classes.....	20
10. Modelo lógico do banco de dados	21
11. Definição dos status e transições do pedido.....	27
12. Diagrama de sequência do fluxo crítico	30
13. Definição da arquitetura e separação em camadas.....	31
14. Definição dos recursos da API	35
15. Endpoints.....	39
16. Definição dos contratos de request e response	59
17. Definição dos parâmetros de rota, consulta e filtros	61
18. Definição dos códigos de status HTTP	62
19. Definição do padrão de respostas e erros	63
20. Plano de testes da API.....	64
21. Implementação, itens conceituais e evidências	70
22. Repositório do projeto.....	75
23. Conclusão	76
Referências.....	77

1. Estudo de Caso

Nome técnico: FastBite é uma rede de lanchonetes composta por uma matriz e as unidades. A matriz é a administração geral da rede, enquanto cada unidade gerencia seu próprio cardápio, estoque e atendimento aos clientes.

Há três canais de vendas na FastBite

- APP (Aplicativo Mobile);
- WEB (Site);
- TOTEM (Autoatendimento).

Para garantir rastreabilidade operacional, todo pedido deve registrar o canal pelo qual foi realizado.

Atualmente, as unidades controlam pedidos e estoque de forma descentralizada, dificultando o acompanhamento das operações e aumentando a problemas de venda de produtos indisponíveis, divergências de estoque e falta de padronização no fluxo dos pedidos.

Há vários objetivos ligados a esse caso, é de suma importância a documentação do sistema, disponibilizar o repositório com os códigos, implantar um programa de fidelização, integrar um serviço de pagamento externo (simulado neste projeto) e garantir conformidade com a Lei Geral de Proteção de Dados (LGPD).

Objetivo

Desenvolver uma API REST para centralizar a gestão da rede de lanchonetes, permitindo que os três canais de atendimento utilizem a base de dados e as regras de negócio.

A API terá autenticação de usuários, controle de acesso por perfis, gerenciamento de pedidos, estoque por unidade, programa de fidelização, integração com pagamento mock e documentação via Swagger/OpenAPI.

Fluxo Principal

O cliente realiza login e escolhe uma unidade da rede. Em seguida, consulta o cardápio e cria um pedido informando obrigatoriamente o campo canalPedido, que pode assumir os valores APP, WEB ou TOTEM.

Antes da confirmação, o sistema verifica a disponibilidade dos produtos em estoque. Caso todos os itens estejam disponíveis, o pedido é registrado com status Aguardando Pagamento e uma solicitação é enviada ao serviço de pagamento mock.

Se o pagamento for aprovado, o pedido segue o fluxo Em Preparo → Pronto → Entregue. Caso seja recusado, o pedido é cancelado.

Após a entrega, o estoque é atualizado, os pontos de fidelidade são creditados ao cliente (quando houver consentimento para participação no programa) e as ações relevantes são registradas em log para auditoria.

Escopo da Solução

Nesta primeira versão do sistema, serão desenvolvidas as funcionalidades essenciais para demonstrar o funcionamento da API. O projeto contará com cadastro e autenticação de usuários, gerenciamento das unidades da rede, cadastro de produtos, controle de estoque e criação de pedidos.

Também será implementado o registro do canal de origem de cada pedido (APP, WEB ou TOTEM), permitindo identificar por qual meio a compra foi realizada. O fluxo de pagamento será representado por uma integração simulada (mock), possibilitando testar tanto pagamentos aprovados quanto recusados.

Além disso, o sistema contará com um programa básico de fidelização, aplicação dos requisitos mínimos de segurança e LGPD, persistência dos dados em banco de dados relacional e documentação da API por meio do Swagger. As funcionalidades que não fazem parte do fluxo principal poderão ser descritas como propostas para versões futuras.

2. Requisitos Funcionais

A API manterá esses requisitos.

Cadastro de Usuários

O sistema deve permitir o cadastro de usuários, armazenando suas informações de forma segura.

Autenticação

O sistema deve permitir que usuários realizem login por meio de autenticação utilizando e-mail e senha.

Controle de Perfis

O sistema deve controlar o acesso às funcionalidades conforme o perfil do usuário (Administrador, Gerente e Cliente).

Gerenciamento de Unidades

O sistema deve permitir o cadastro, consulta, atualização e remoção das unidades da rede.

Gerenciamento de Produtos

O sistema deve permitir o cadastro, consulta, atualização e remoção dos produtos disponíveis em cada unidade.

Consulta de Cardápio

O sistema deve permitir a consulta do cardápio de uma unidade específica.

Controle de Estoque

O sistema deve registrar entradas e saídas de produtos, mantendo o estoque atualizado por unidade.

Validação de Estoque

O sistema deve verificar a disponibilidade dos produtos antes da criação de um pedido.

Criação de Pedidos

O sistema deve permitir a criação de pedidos contendo um ou mais produtos.

Registro do Canal do Pedido

O sistema deve exigir o preenchimento do campo canalPedido, permitindo apenas os valores APP, WEB e TOTEM.

Consulta de Pedidos

O sistema deve permitir consultar pedidos, inclusive filtrando-os pelo canal de origem.

Atualização de Status do Pedido

O sistema deve permitir a atualização do status do pedido conforme o fluxo operacional:

- Aguardando Pagamento;
- Em Preparo;
- Pronto;
- Entregue;
- Cancelado.

Cancelamento de Pedidos

O sistema deve permitir o cancelamento de pedidos conforme as regras de negócio.

Integração com Pagamento Mock

O sistema deve simular o envio de uma solicitação para um serviço externo de pagamento, registrando o retorno da operação.

Programa de Fidelização

O sistema deve registrar pontos de fidelidade para clientes que concluírem pedidos com pagamento aprovado e que tenham consentido participar do programa.

Registro de Logs

O sistema deve registrar em log ações importantes, como criação e cancelamento de pedidos, alterações de status e movimentações de estoque.

3. Requisitos Não Funcionais

Segurança

A API deverá utilizar autenticação baseada em JWT para garantir que apenas usuários autorizados tenham acesso às funcionalidades do sistema. As permissões deverão ser controladas conforme o perfil de cada usuário.

Proteção de Dados (LGPD)

O sistema deverá coletar apenas os dados necessários para a execução de suas funcionalidades, armazenando-os de forma segura e respeitando os princípios da Lei Geral de Proteção de Dados (LGPD). A participação no programa de fidelização dependerá do consentimento do cliente.

Armazenamento Seguro

As senhas dos usuários deverão ser armazenadas utilizando algoritmos de hash, impedindo que sejam gravadas em texto puro no banco de dados.

Persistência dos Dados

Todas as informações da aplicação deverão ser armazenadas em um banco de dados relacional, utilizando ORM e migrations para garantir consistência entre o modelo de dados e a implementação.

Arquitetura

A aplicação deverá seguir uma arquitetura em camadas (Domain, Application, Infrastructure e API), promovendo a separação de responsabilidades e facilitando a manutenção do sistema.

Documentação da API

Todos os endpoints deverão estar documentados por meio do Swagger/OpenAPI, contendo exemplos de requisições, respostas e códigos de status.

Padronização das Respostas

A API deverá retornar respostas padronizadas para operações de sucesso e erro, utilizando códigos de status HTTP adequados e mensagens claras para facilitar a integração com os diferentes canais.

Desempenho

A API deverá responder às requisições de forma eficiente, mantendo um bom desempenho mesmo com múltiplas solicitações simultâneas provenientes dos canais APP, WEB e TOTEM.

Auditoria

O sistema deverá registrar em log as principais ações realizadas pelos usuários, como autenticação, criação e cancelamento de pedidos, alterações de status e movimentações de estoque, permitindo a rastreabilidade das operações.

Manutenibilidade

O código deverá ser organizado de forma modular, seguindo boas práticas de desenvolvimento e nomenclatura, facilitando futuras manutenções e evoluções do sistema.

Portabilidade

A aplicação deverá possuir um processo de instalação e execução documentado no README, incluindo instruções para configuração do ambiente, banco de dados, variáveis de ambiente e execução das migrations, permitindo que qualquer avaliador consiga executar a API.

4. Priorização das Funcionalidades

Para o desenvolvimento deste projeto, as funcionalidades foram priorizadas de acordo com sua importância para o funcionamento da aplicação. O foco foi implementar primeiro os recursos essenciais, garantindo um fluxo completo de utilização da API.

As funcionalidades consideradas **essenciais** são o cadastro e autenticação de usuários, o controle de acesso por perfis, o gerenciamento das unidades, produtos e estoque, a criação de pedidos, o registro obrigatório do **canalPedido**, a integração com o serviço de pagamento mock e a atualização do status dos pedidos. Esses recursos compõem o fluxo principal da aplicação e são indispensáveis para demonstrar o funcionamento do sistema.

Como funcionalidades **importantes**, foram incluídos o programa de fidelização, os registros de logs e auditoria, a documentação da API com Swagger/OpenAPI e a aplicação dos requisitos mínimos da LGPD. Embora não impeçam o funcionamento do sistema, agregam segurança, rastreabilidade e qualidade à solução.

Por fim, funcionalidades como promoções e campanhas foram tratadas apenas de forma conceitual, ficando previstas para futuras evoluções do sistema. Essa decisão permitiu concentrar o desenvolvimento nos requisitos obrigatórios e garantir a entrega de uma solução completa, estável e alinhada aos objetivos do projeto.

5. Regras de Negócio

As regras de negócio definem as condições que devem ser respeitadas para garantir o funcionamento correto da aplicação e a consistência dos dados.

Cadastro de Usuários

- O cadastro deverá exigir informações obrigatórias, como nome, e-mail e senha.
- O e-mail deverá ser único no sistema.
- A senha será armazenada utilizando hash criptográfico.

Autenticação

- Apenas usuários cadastrados poderão acessar as funcionalidades protegidas da API.
- O acesso às funcionalidades será controlado conforme o perfil do usuário (Administrador, Gerente ou Cliente).

Unidades

- Cada unidade possuirá seu próprio cardápio e controle de estoque.
- Produtos indisponíveis em uma unidade não poderão ser vendidos por ela.

Produtos

- Um produto deverá estar vinculado a uma unidade.
- Apenas produtos ativos poderão ser exibidos no cardápio.

Estoque

- Toda entrada ou saída de produtos deverá atualizar o estoque da unidade correspondente.
- Não será permitida a criação de pedidos com quantidade superior ao estoque disponível.

Pedidos

- Todo pedido deverá possuir pelo menos um item.

- O campo **canalPedido** será obrigatório e aceitará apenas os valores **APP**, **WEB** ou **TOTEM**.
- O pedido será criado inicialmente com o status **Aguardando Pagamento**.
- O pedido poderá ser cancelado apenas enquanto ainda não tiver sido entregue.

Pagamento

- Após a criação do pedido, será realizada uma solicitação ao serviço de pagamento mock.
- Se o pagamento for aprovado, o pedido seguirá para o status **Em Preparo**.
- Se o pagamento for recusado, o pedido será automaticamente cancelado.

Atualização de Status

Os pedidos deverão seguir o seguinte fluxo:

Aguardando Pagamento → Em Preparo → Pronto → Entregue

Não será permitido alterar o status para uma etapa anterior ou ignorar etapas do fluxo.

Programa de Fidelização

- Apenas clientes que autorizarem a participação no programa poderão acumular pontos.
- Os pontos serão creditados somente após a confirmação do pagamento e a conclusão do pedido.

Logs e Auditoria

- O sistema deverá registrar ações como login, criação e cancelamento de pedidos, alterações de status e movimentações de estoque.
- Os registros de auditoria deverão armazenar a data, a ação realizada e o usuário responsável.

Segurança e LGPD

- A API não deverá expor dados sensíveis, como senhas, em suas respostas.
- O sistema deverá coletar apenas os dados necessários para o funcionamento das funcionalidades implementadas.
- O consentimento do cliente para participar do programa de fidelização deverá ser registrado antes da utilização de seus dados para essa finalidade.

6. Identificação dos atores e perfis de acesso.

O sistema será utilizado por pessoas com responsabilidades diferentes dentro da rede de lanchonetes. Por esse motivo, cada usuário terá acesso somente às funções necessárias para realizar suas atividades. Essa divisão reduz o risco de alterações indevidas e facilita a identificação de quem realizou cada ação.

Os pedidos poderão ser feitos exclusivamente pelos canais APP, WEB e TOTEM. Esses canais representam a origem do pedido, enquanto os perfis determinam quais ações cada usuário poderá realizar no sistema.

Cliente

O cliente é quem utiliza o sistema para consultar o cardápio de uma unidade e realizar pedidos. Ele poderá criar uma conta, fazer login, escolher os produtos, informar o canal do pedido, solicitar o pagamento, acompanhar o andamento e cancelar o pedido quando as regras permitirem.

Também poderá participar do programa de fidelidade, desde que registre seu consentimento para o tratamento dos dados necessários. O cliente terá acesso apenas às próprias informações e aos pedidos vinculados à sua conta.

Atendente

O atendente auxilia o cliente e acompanha os pedidos da unidade em que trabalha. Ele poderá consultar pedidos, verificar seus detalhes e ajudar em situações como retirada, dúvidas ou solicitação de cancelamento.

Apesar da participação do atendente na operação, o sistema não utilizará BALCAO como canal de pedido. Todo pedido continuará sendo identificado como originado por APP, WEB ou TOTEM.

Cozinha

A equipe da cozinha terá acesso aos pedidos que já passaram pela validação e tiveram o pagamento aprovado. Sua principal responsabilidade será acompanhar a fila de produção e atualizar o pedido para os status EM_PREPARO e PRONTO.

Esse perfil não poderá cadastrar usuários, alterar preços, movimentar estoque manualmente ou administrar unidades.

Gerente

O gerente será responsável pela operação de uma unidade específica. Ele poderá gerenciar o cardápio local, definir preços, consultar o estoque, registrar entradas e saídas, acompanhar pedidos e configurar promoções relacionadas à sua unidade.

Também poderá consultar o histórico das operações necessárias para acompanhar o funcionamento da unidade. Entretanto, não terá permissão para administrar outras unidades ou modificar as configurações gerais da rede.

Administrador

O administrador terá o maior nível de acesso. Ele será responsável por cadastrar unidades, gerenciar usuários e perfis, consultar dados de toda a rede e acessar os registros de auditoria.

Esse perfil também poderá realizar configurações gerais e corrigir situações administrativas. Por possuir acesso a funções sensíveis, suas ações deverão ser registradas no log de auditoria.

Serviço externo de pagamento

O serviço externo de pagamento será representado por uma integração mock. Ele receberá a solicitação de pagamento e retornará uma resposta simulada, indicando se a operação foi aprovada, recusada ou permaneceu pendente.

A aplicação armazenará os dados relevantes da solicitação e da resposta para permitir o acompanhamento do fluxo. Nenhum pagamento real será processado.

Resumo das permissões

Perfil ou ator

Cliente: Consultar cardápio, criar pedido, acompanhar status, solicitar cancelamento e utilizar a fidelidade.

Atendente: Consultar e acompanhar pedidos da própria unidade e auxiliar no atendimento

Cozinha: Visualizar pedidos aprovados e atualizar para EM_PREPARO e PRONTO

Gerente: Gerenciar cardápio, preços, estoque, pedidos e promoções da própria unidade

Administrador: Gerenciar toda a rede, unidades, usuários, perfis e registros de auditoria

Pagamento mock: Receber solicitações e retornar o resultado simulado do pagamento

O controle dessas permissões será realizado por autenticação via token JWT e autorização baseada em perfis. As senhas serão armazenadas utilizando hash seguro, e ações consideradas sensíveis, como alterações de estoque, preços, usuários e permissões, serão registradas para fins de auditoria.

7. Definição do fluxo crítico do sistema.

O fluxo mais importante do sistema começa quando o cliente escolhe uma unidade, consulta o cardápio e adiciona os produtos ao pedido. Na hora de finalizar, ele precisa informar por qual canal está fazendo a compra: APP, WEB ou TOTEM.

Antes de aceitar o pedido, o sistema confere se a unidade está ativa, se os produtos fazem parte do cardápio daquela unidade e se existe estoque suficiente para atender às quantidades solicitadas. Se estiver tudo certo, os itens são reservados no estoque e o pedido é criado com o status AGUARDANDO_PAGAMENTO.

Em seguida, o sistema envia a solicitação para o pagamento mock, que simula o funcionamento de um serviço externo. Tanto os dados enviados quanto a resposta recebida ficam registrados no banco. Se o pagamento for aprovado, a reserva do estoque é confirmada como saída e o pedido pode seguir para o preparo. Se o pagamento for recusado, o pedido é cancelado e os itens reservados voltam para o estoque.

Depois da aprovação, a cozinha atualiza o pedido para EM_PREPARO e, quando terminar, para PRONTO. Após a retirada ou entrega, o status muda para ENTREGUE. Cada mudança fica salva no histórico para que seja possível saber quando ela aconteceu e quem realizou a ação.

Quando o pedido é entregue, o cliente pode receber pontos no programa de fidelidade, desde que tenha uma conta de fidelidade e um consentimento LGPD ativo. Em caso de cancelamento, o sistema também deve devolver qualquer quantidade reservada e impedir que pontos sejam gerados.

De forma resumida, o fluxo principal funciona assim:

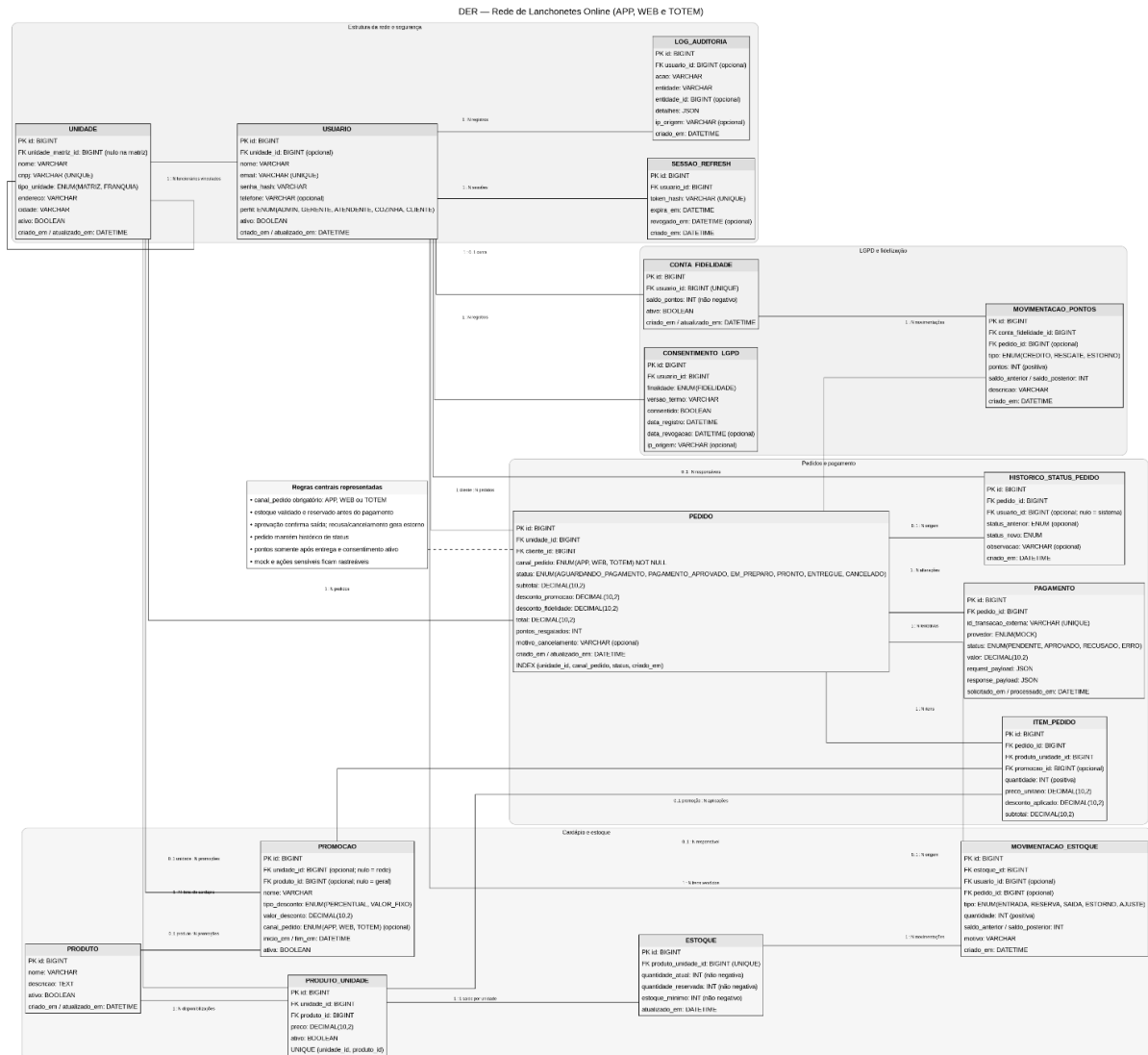
Criação do pedido → Validação dos itens → Reserva do estoque → Pagamento mock → Preparo → Pedido pronto → Entrega → Pontos de fidelidade

Os principais fluxos de erro acontecem quando o canal do pedido é inválido, algum produto não pertence ao cardápio da unidade, o estoque é insuficiente, o pagamento é recusado ou alguém tenta atualizar o pedido para um status que não faz sentido naquele momento.

8. Diagrama de Entidade-Relacionamento

Antes de iniciar o desenvolvimento da API, foi elaborado um Diagrama de Entidade-Relacionamento para definir como os dados seriam organizados no banco de dados. A modelagem foi construída considerando todas as funcionalidades previstas para o sistema, como o controle de pedidos, estoque, pagamentos, usuários e fidelização. Dessa forma, o diagrama serve como base para a implementação da aplicação e ajuda a garantir que todas as partes do sistema estejam integradas de forma consistente.

Figura 1 – Diagrama de DER



A Figura X apresenta o modelo de dados desenvolvido para sustentar as principais funcionalidades da API. Fonte: Autoria própria (2026).

Arquivo:

<https://imgur.com/a/pBp8Xld>

https://drive.google.com/drive/folders/1pHfzpLu4w-30X2Rj0FdjYI-G4RTqZ99?usp=drive_link

Principais decisões do modelo

O DER foi montado para representar o funcionamento completo da rede de lanchonetes, desde o cadastro das unidades e dos produtos até a entrega do pedido e a geração dos pontos de fidelidade. A ideia foi evitar que o banco servisse apenas para guardar informações básicas. Ele também precisa sustentar as regras de negócio, manter o histórico das operações e permitir que a API trabalhe com dados consistentes.

Ao todo, o modelo possui 16 entidades. Elas foram separadas de acordo com suas responsabilidades para evitar informações repetidas e facilitar futuras alterações no sistema.

Organização das unidades

A entidade UNIDADE representa tanto a matriz quanto as demais unidades da rede. Para isso, foi criado um autorrelacionamento, em que uma unidade pode estar vinculada a outra como sua unidade superior. Dessa forma, a matriz pode possuir várias unidades vinculadas, sem a necessidade de criar uma entidade diferente para cada tipo de estabelecimento.

Essa estrutura se conecta diretamente aos endpoints de unidades, responsáveis por cadastrar, atualizar e consultar os estabelecimentos da rede. Ela também permite que os demais recursos, como cardápio, estoque, pedidos e promoções, sejam organizados por unidade.

Usuários, perfis e autenticação

A entidade USUARIO reúne clientes e funcionários. A diferença entre eles é definida pelo perfil de acesso, como cliente, atendente, cozinha, gerente ou administrador. O vínculo com uma unidade é opcional, pois alguns usuários pertencem a uma unidade específica, enquanto outros, como clientes e administradores gerais, não precisam desse vínculo.

Essa decisão evita a criação de uma tabela diferente para cada tipo de usuário e simplifica a autenticação. Os endpoints de cadastro, login e gerenciamento de usuários trabalham com essa mesma entidade, enquanto as permissões são verificadas de acordo com o perfil registrado.

A entidade SESSAO_REFRESH foi separada de USUARIO para armazenar as sessões de renovação do token. Com isso, o sistema consegue controlar refresh tokens, expiração e logout sem guardar essas informações diretamente no cadastro do usuário. Essa parte do modelo dá suporte aos endpoints /auth/login, /auth/refresh e /auth/logout.

Produtos e cardápio por unidade

Os produtos foram separados entre as entidades PRODUTO e PRODUTO_UNIDADE. A entidade PRODUTO guarda as informações gerais, como

nome, descrição e categoria. Já PRODUTO_UNIDADE informa em quais unidades o produto está disponível e qual é o preço praticado em cada uma delas.

Essa separação foi necessária porque um mesmo produto pode ser vendido em várias unidades, mas não necessariamente pelo mesmo preço ou com a mesma disponibilidade.

Na API, essa estrutura permite ter endpoints para o cadastro geral dos produtos e outros voltados à consulta do cardápio de uma unidade. Quando o cliente consulta o cardápio, o sistema busca apenas os produtos ativos e disponíveis naquela unidade.

Estoque e movimentações

O estoque foi modelado por unidade e por item do cardápio. Cada registro de ESTOQUE pertence a uma combinação de produto e unidade, impedindo que duas unidades compartilhem o mesmo saldo por engano.

Além do saldo atual, foi criada a entidade MOVIMENTACAO_ESTOQUE. Ela registra entradas, reservas, saídas e estornos, mantendo o motivo, a quantidade e o momento de cada movimentação. Essa separação é importante porque apenas alterar o saldo não seria suficiente para explicar como ele chegou àquela quantidade.

Os endpoints de estoque utilizam essas entidades para consultar o saldo de uma unidade e registrar movimentações. Durante a criação de um pedido, a API verifica o estoque antes de continuar. Se a quantidade não for suficiente, o pedido não é aceito e a API retorna um erro de regra de negócio. Se houver disponibilidade, os itens são reservados antes da solicitação de pagamento.

Pedido, itens e canal de origem

A entidade PEDIDO concentra as informações principais da compra, como cliente, unidade, canal de origem, status e valores. Cada pedido possui um ou mais registros em ITEM_PEDIDO, nos quais ficam armazenados os produtos, as quantidades e os valores usados no momento da compra.

Os valores são registrados no item porque o preço de um produto pode mudar depois. Dessa forma, um pedido antigo continua mostrando o valor correto que foi cobrado quando ele foi realizado.

O campo canalPedido é obrigatório e aceita somente APP, WEB ou TOTEM. Essa restrição permite identificar de onde cada pedido veio e facilita a rastreabilidade entre os canais. Ela também aparece diretamente no contrato do endpoint de criação:

```
{  
  "canalPedido": "TOTEM",  
  "clienteId": 123,
```

```
"unidadeId": 1,  
"itens": [  
  {  
    "produtoId": 10,  
    "quantidade": 2  
  }  
],  
"formaPagamento": "MOCK"  
}
```

Caso o canal não seja informado ou tenha um valor diferente dos permitidos, a API deve recusar a criação com status 400 ou 422 e uma mensagem padronizada. O mesmo campo também poderá ser usado como filtro na consulta de pedidos:

GET /pedidos?canalPedido=APP&status=AGUARDANDO_PAGAMENTO

Histórico de status

O status atual fica registrado em PEDIDO, mas cada mudança também gera um registro em HISTORICO_STATUS_PEDIDO. Essa decisão permite consultar o estado atual de forma rápida sem perder o histórico do que aconteceu.

O pedido começa como AGUARDANDO_PAGAMENTO. Após a aprovação, pode avançar para PAGAMENTO_APROVADO, EM_PREPARO, PRONTO e ENTREGUE. Em caso de pagamento recusado ou cancelamento, passa para CANCELADO.

O endpoint de atualização de status deve seguir essa ordem. Por exemplo, um pedido não poderá mudar diretamente de AGUARDANDO_PAGAMENTO para PRONTO. Se uma transição inválida for solicitada, a API deverá retornar um erro de regra de negócio. Cada alteração válida fica registrada no histórico, junto com o usuário responsável e o momento da ação.

Pagamento mock

A entidade PAGAMENTO foi separada de PEDIDO porque um pedido pode possuir mais de uma tentativa de pagamento. Ela guarda o provedor utilizado, o status da tentativa e os dados enviados e recebidos durante a integração mock.

Mesmo sem utilizar um serviço real, o sistema representa o fluxo de uma integração externa. A API envia a solicitação, recebe uma resposta simulada e registra os dois payloads. Se o pagamento for aprovado, a reserva do estoque é confirmada como saída e o pedido pode seguir para o preparo. Se for recusado, o pedido é cancelado e a quantidade reservada é devolvida ao estoque.

Com isso, os endpoints de pagamento não ficam isolados do restante do sistema. A resposta do pagamento interfere diretamente no status do pedido e nas movimentações de estoque.

Fidelidade e consentimento LGPD

O programa de fidelidade foi dividido entre `CONTA_FIDELIDADE` e `MOVIMENTACAO_PONTOS`. A conta guarda o saldo atual do cliente, enquanto as movimentações registram cada entrada ou resgate de pontos. Essa separação funciona de forma parecida com o controle de estoque: existe um saldo para consulta rápida, mas também existe um histórico que explica todas as alterações.

A conta de fidelidade é opcional, pois nem todo usuário precisa participar do programa. Além disso, os pontos só podem ser gerados depois que o pedido chegar ao status `ENTREGUE` e quando o cliente possuir um consentimento ativo em `CONSENTIMENTO_LGPD`.

A entidade de consentimento foi criada para registrar de forma clara a autorização do cliente, sua finalidade e a situação atual. Dessa maneira, os endpoints de fidelidade precisam verificar o consentimento antes de gerar pontos, ligando a regra da LGPD diretamente ao funcionamento da API.

Promoções

A entidade `PROMOCAO` permite representar campanhas gerais ou direcionadas a uma unidade, produto ou canal específico. Essa flexibilidade evita a criação de várias tabelas para diferentes tipos de promoção.

Antes de aplicar um desconto, a API deverá verificar se a promoção está ativa, se está dentro do período válido e se pode ser usada na unidade, no produto e no canal informados. Assim, a regra da promoção é aplicada de forma consistente no cálculo do pedido.

Logs e auditoria

A entidade `LOG_AUDITORIA` foi incluída para registrar ações consideradas sensíveis, como alterações de preço, movimentações manuais de estoque, mudanças de permissões e atualizações importantes no pedido.

Esse registro permite identificar qual usuário realizou a ação, quando ela ocorreu e qual recurso foi afetado. A auditoria complementa os históricos específicos de estoque, pedido e fidelidade, oferecendo uma visão mais ampla das operações realizadas pela API.

Coerência entre o DER e a API

As entidades do DER foram pensadas para acompanhar os principais recursos da API. `USUARIO` e `SESSAO_REFRESH` sustentam a autenticação; `UNIDADE`, `PRODUTO` e `PRODUTO_UNIDADE` formam a base do cardápio; `ESTOQUE` e

MOVIMENTACAO_ESTOQUE controlam a disponibilidade; PEDIDO, ITEM_PEDIDO e HISTORICO_STATUS_PEDIDO formam o fluxo principal; e PAGAMENTO representa a integração mock.

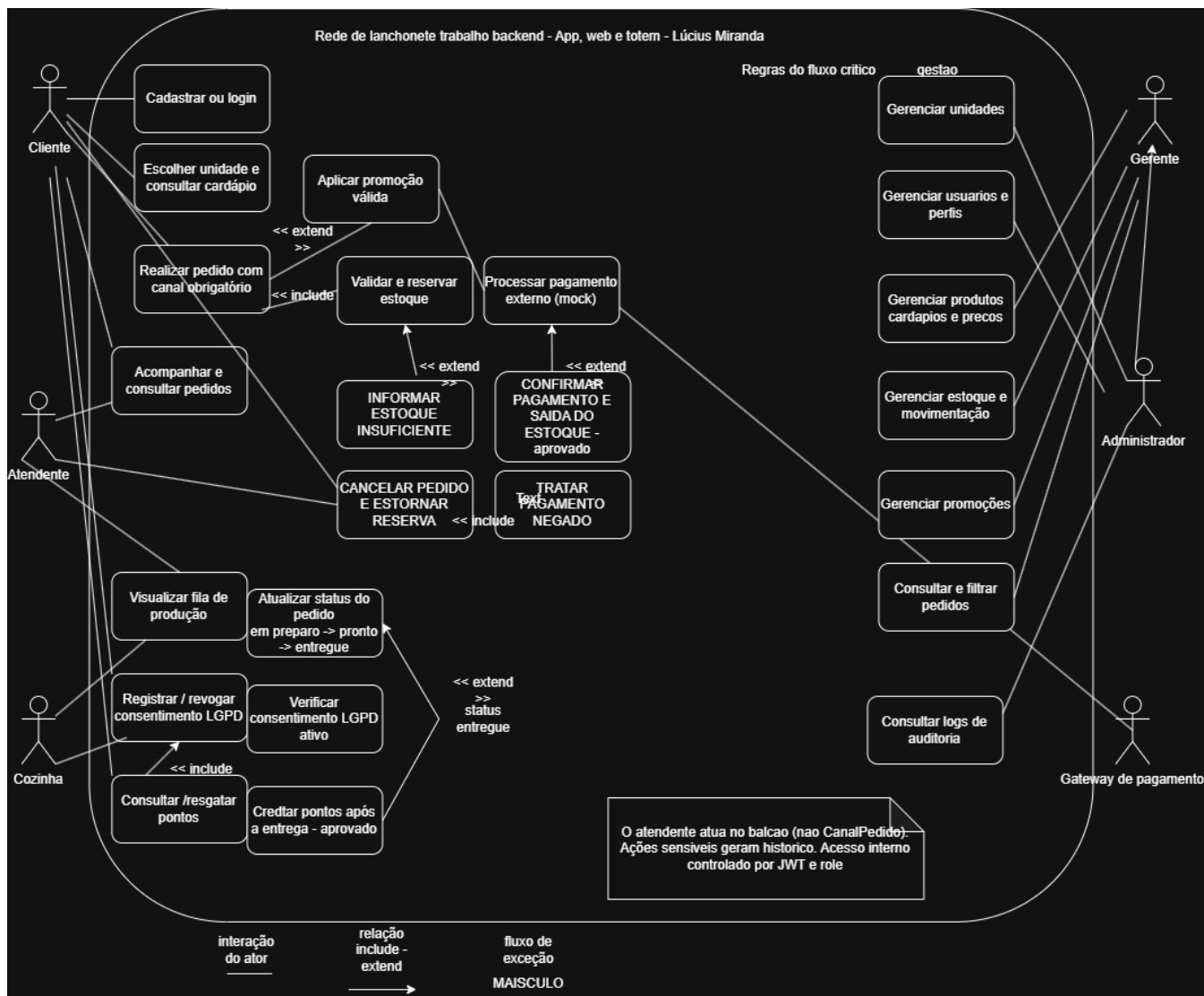
Da mesma forma, CONSENTIMENTO_LGPD, CONTA_FIDELIDADE e MOVIMENTACAO_PONTOS sustentam o programa de fidelidade, enquanto PROMOCAO e LOG_AUDITORIA completam as regras comerciais e de segurança.

Com essa organização, cada endpoint possui uma base clara no modelo de dados. As chaves estrangeiras garantem os relacionamentos entre os recursos, enquanto as restrições do banco e as validações da aplicação impedem situações inválidas, como criar pedidos sem cliente, unidade ou itens, vender produtos sem estoque, informar um canal inexistente, avançar o pedido para um status incorreto ou gerar pontos sem consentimento ativo.

8. 1 Diagrama de casos de uso.

O diagrama de casos de uso apresenta os principais atores que interagem com a API da rede de lanchonetes e as funcionalidades disponíveis para cada perfil. Também são representados o fluxo principal de realização do pedido e os fluxos alternativos provocados por estoque insuficiente ou pagamento negado.

Figura 2 - Diagrama de casos de uso



Fonte: Autoria própria (2026).

Arquivo:

- <https://imgur.com/a/NkX3ST6>
- https://drive.google.com/drive/folders/1pHfzpLul4w-30X2Rj0FdjYl-G4RTqZ99?usp=drive_link

9. Diagrama de Classes

O diagrama de classes apresenta a estrutura dos principais objetos que fazem parte da API da rede de lanchonetes. Diferentemente do DER, que representa principalmente a organização do banco de dados, este diagrama também mostra atributos, comportamentos, tipos enumerados e as responsabilidades de cada classe.

As classes foram organizadas em quatro grupos: identidade e acesso; rede, catálogo e estoque; pedidos e pagamentos; fidelidade, promoções e auditoria. Essa organização mantém a ligação entre o modelo de dados, as regras de negócio e os casos de uso apresentados anteriormente.

Figura 3 – Diagrama de classes

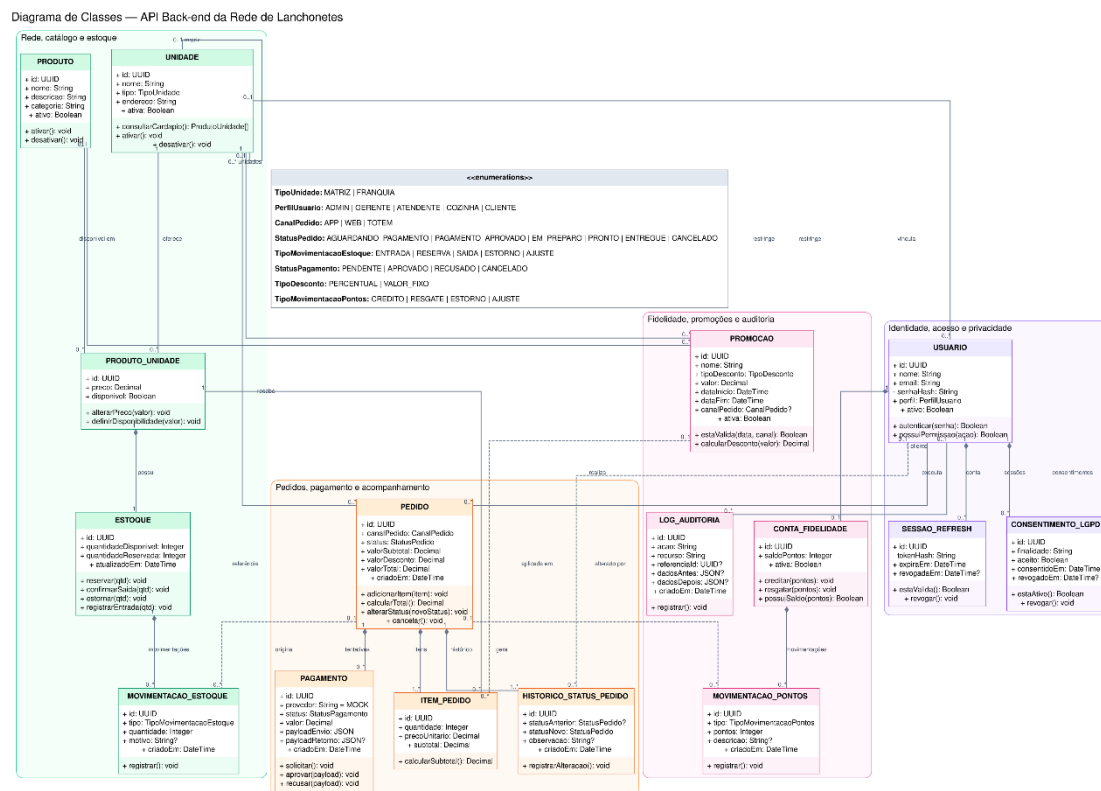


Figura 3 – Diagrama de classes da API para a rede de lanchonetes

Fonte: Autoria própria (2026).

Arquivo:

- <https://imgur.com/a/34wrszN>
- https://drive.google.com/drive/folders/1pHfzpLu4w-30X2Rj0FdjYl-G4RTqZ99?usp=drive_link

10. Modelo lógico do banco de dados

Unidades, usuários

UNIDADE

- id: UUID [PK]
- unidade_matriz_id: UUID [FK → UNIDADE.id, opcional]
- nome: VARCHAR(120) [NOT NULL]
- tipo: TIPO_UNIDADE [NOT NULL]
- endereco: TEXT [NOT NULL]
- ativa: BOOLEAN [NOT NULL, DEFAULT TRUE]
- criado_em: TIMESTAMPTZ [NOT NULL]
- atualizado_em: TIMESTAMPTZ [NOT NULL]

unidade_matriz_id

USUARIO

- id: UUID [PK]
- unidade_id: UUID [FK → UNIDADE.id, opcional]
- nome: VARCHAR(120) [NOT NULL]
- email: VARCHAR(160) [NOT NULL, UNIQUE]
- senha_hash: VARCHAR(255) [NOT NULL]
- perfil: PERFIL_USUARIO [NOT NULL]
- ativo: BOOLEAN [NOT NULL, DEFAULT TRUE]
- criado_em: TIMESTAMPTZ [NOT NULL]
- atualizado_em: TIMESTAMPTZ [NOT NULL]

SESSAO_REFRESH

- id: UUID [PK]
- usuario_id: UUID [FK → USUARIO.id, NOT NULL]
- token_hash: VARCHAR(255) [NOT NULL, UNIQUE]

- expira_em: TIMESTAMPTZ [NOT NULL]
- revogada_em: TIMESTAMPTZ [opcional]
- criado_em: TIMESTAMPTZ [NOT NULL]

PRODUTO

- id: UUID [PK]
- nome: VARCHAR(120) [NOT NULL]
- descricao: TEXT [opcional]
- categoria: VARCHAR(80) [NOT NULL]
- ativo: BOOLEAN [NOT NULL, DEFAULT TRUE]
- criado_em: TIMESTAMPTZ [NOT NULL]
- atualizado_em: TIMESTAMPTZ [NOT NULL]

PRODUTO_UNIDADE

- id: UUID [PK]
- unidade_id: UUID [FK → UNIDADE.id, NOT NULL]
- produto_id: UUID [FK → PRODUTO.id, NOT NULL]
- preco: NUMERIC(10,2) [NOT NULL]
- disponivel: BOOLEAN [NOT NULL, DEFAULT TRUE]
- criado_em: TIMESTAMPTZ [NOT NULL]
- atualizado_em: TIMESTAMPTZ [NOT NULL]

Restrição UNIQUE:

- unidade_id + produto_id

ESTOQUE (cardápio)

- id: UUID [PK]
- produto_unidade_id: UUID [FK → PRODUTO_UNIDADE.id, NOT NULL, UNIQUE]
- quantidade_disponivel: INTEGER [NOT NULL, DEFAULT 0]

- quantidade_reservada: INTEGER [NOT NULL, DEFAULT 0]
- atualizado_em: TIMESTAMPTZ [NOT NULL]

MOVIMENTACAO_ESTOQUE

- id: UUID [PK]
- estoque_id: UUID [FK → ESTOQUE.id, NOT NULL]
- pedido_id: UUID [FK → PEDIDO.id, opcional]
- tipo: TIPO_MOVIMENTACAO_ESTOQUE [NOT NULL]
- quantidade: INTEGER [NOT NULL]
- motivo: VARCHAR(255) [opcional]
- criado_em: TIMESTAMPTZ [NOT NULL]

Obs: ENTRADA, RESERVA, SAIDA, ESTORNO ou AJUSTE

PEDIDO

- id: UUID [PK]
- cliente_id: UUID [FK → USUARIO.id, NOT NULL]
- unidade_id: UUID [FK → UNIDADE.id, NOT NULL]
- canal_pedido: CANAL_PEDIDO [NOT NULL]
- status: STATUS_PEDIDO [NOT NULL]
- valor_subtotal: NUMERIC(10,2) [NOT NULL]
- valor_desconto: NUMERIC(10,2) [NOT NULL, DEFAULT 0]
- valor_total: NUMERIC(10,2) [NOT NULL]
- criado_em: TIMESTAMPTZ [NOT NULL]
- atualizado_em: TIMESTAMPTZ [NOT NULL]

Obs: app, web e totem.

ITEM_PEDIDO

- id: UUID [PK]

- pedido_id: UUID [FK → PEDIDO.id, NOT NULL]
- produto_unidade_id: UUID [FK → PRODUTO_UNIDADE.id, NOT NULL]
- promocao_id: UUID [FK → PROMOCAO.id, opcional]
- quantidade: INTEGER [NOT NULL]
- preco_unitario: NUMERIC(10,2) [NOT NULL]
- subtotal: NUMERIC(10,2) [NOT NULL]

HISTORICO_STATUS_PEDIDO

- id: UUID [PK]
- pedido_id: UUID [FK → PEDIDO.id, NOT NULL]
- usuario_id: UUID [FK → USUARIO.id, opcional]
- status_anterior: STATUS_PEDIDO [opcional]
- status_novo: STATUS_PEDIDO [NOT NULL]
- observacao: VARCHAR(255) [opcional]
- criado_em: TIMESTAMPTZ [NOT NULL]

AGUARDANDO_PAGAMENTO

PAGAMENTO_APROVADO

EM_PREPARO

PRONTO

ENTREGUE

CANCELADO

PAGAMENTO

- id: UUID [PK]
- pedido_id: UUID [FK → PEDIDO.id, NOT NULL]
- provedor: VARCHAR(30) [NOT NULL, DEFAULT 'MOCK']
- status: STATUS_PAGAMENTO [NOT NULL]
- valor: NUMERIC(10,2) [NOT NULL]

- payload_envio: JSONB [NOT NULL]
- payload_retorno: JSONB [opcional]
- criado_em: TIMESTAMPTZ [NOT NULL]
- atualizado_em: TIMESTAMPTZ [NOT NULL]

CONSENTIMENTO_LGPD

- id: UUID [PK]
- usuario_id: UUID [FK → USUARIO.id, NOT NULL]
- finalidade: VARCHAR(160) [NOT NULL]
- aceito: BOOLEAN [NOT NULL]
- consentido_em: TIMESTAMPTZ [NOT NULL]
- revogado_em: TIMESTAMPTZ [opcional]

CONTA_FIDELIDADE

- id: UUID [PK]
- usuario_id: UUID [FK → USUARIO.id, NOT NULL, UNIQUE]
- saldo_pontos: INTEGER [NOT NULL, DEFAULT 0]
- ativa: BOOLEAN [NOT NULL, DEFAULT TRUE]
- criado_em: TIMESTAMPTZ [NOT NULL]
- atualizado_em: TIMESTAMPTZ [NOT NULL]

MOVIMENTACAO_PONTOS

- id: UUID [PK]
- conta_fidelidade_id: UUID [FK → CONTA_FIDELIDADE.id, NOT NULL]
- pedido_id: UUID [FK → PEDIDO.id, opcional]
- tipo: TIPO_MOVIMENTACAO_PONTOS [NOT NULL]
- pontos: INTEGER [NOT NULL]
- descricao: VARCHAR(255) [opcional]
- criado_em: TIMESTAMPTZ [NOT NULL]

PROMOCAO

- id: UUID [PK]
- unidade_id: UUID [FK → UNIDADE.id, opcional]
- produto_id: UUID [FK → PRODUTO.id, opcional]
- nome: VARCHAR(120) [NOT NULL]
- tipo_desconto: TIPO_DESCONTO [NOT NULL]
- valor: NUMERIC(10,2) [NOT NULL]
- data_inicio: TIMESTAMPTZ [NOT NULL]
- data_fim: TIMESTAMPTZ [NOT NULL]
- canal_pedido: CANAL_PEDIDO [opcional]
- ativa: BOOLEAN [NOT NULL, DEFAULT TRUE]
- criado_em: TIMESTAMPTZ [NOT NULL]

LOG_AUDITORIA

- id: UUID [PK]
- usuario_id: UUID [FK → USUARIO.id, opcional]
- acao: VARCHAR(100) [NOT NULL]
- recurso: VARCHAR(100) [NOT NULL]
- referencia_id: UUID [opcional]
- dados_antes: JSONB [opcional]
- dados_depois: JSONB [opcional]
- criado_em: TIMESTAMPTZ [NOT NULL]

Essa tabela registra alterações sensíveis, como mudanças de preço, estoque, permissões e status de pedidos.

Restrições de integridade

O banco vai ter:

- E-mail de usuário único;

- Preços e valores monetários não negativos;
- Quantidades de estoque e pontos não negativas;
- Quantidade do item maior que zero;
- Um único produto por unidade no cardápio;
- Um único estoque para cada PRODUTO_UNIDADE;
- data_fim posterior a data_inicio;
- Desconto percentual entre 0 e 100;
- Pedido com cliente, unidade, itens e canal válido;
- Fidelidade com apenas uma conta por usuário;
- Proibição de crédito duplicado de pontos para o mesmo pedido;
- Preservação dos pedidos e registros históricos, mesmo quando usuários ou produtos forem desativados.

Figura 4:Conexão com API (orientada e auxiliada com IA)

Figura 4:

Conexão com a API	
Recurso da API	Tabelas principais
/auth	USUARIO , SESSAO_REFRESH
/unidades	UNIDADE
/produtos e cardápio	PRODUTO , PRODUTO_UNIDADE
/estoque	ESTOQUE , MOVIMENTACAO_ESTOQUE
/pedidos	PEDIDO , ITEM_PEDIDO , HISTORICO_STATUS_PEDIDO
/pagamentos	PAGAMENTO
/fidelidade	CONSENTIMENTO_LGPD , CONTA_FIDELIDADE , MOVIMENTACAO_PONTOS
/promocoes	PROMOCAO
Auditoria	LOG_AUDITORIA

11. Definição dos status e transições do pedido

Status do pedido

AGUARDANDO_PAGAMENTO

É o primeiro status do pedido. Ele é definido depois que os itens são validados e o estoque é reservado. Nesse momento, o sistema aguarda o resultado do pagamento mock.

PAGAMENTO_APROVADO

Indica que o gateway mock aprovou o pagamento. A reserva do estoque é confirmada como saída e o pedido fica disponível para a cozinha.

EM_PREPARO

Indica que a cozinha iniciou a preparação do pedido. Depois desse ponto, o cancelamento comum não será permitido.

PRONTO

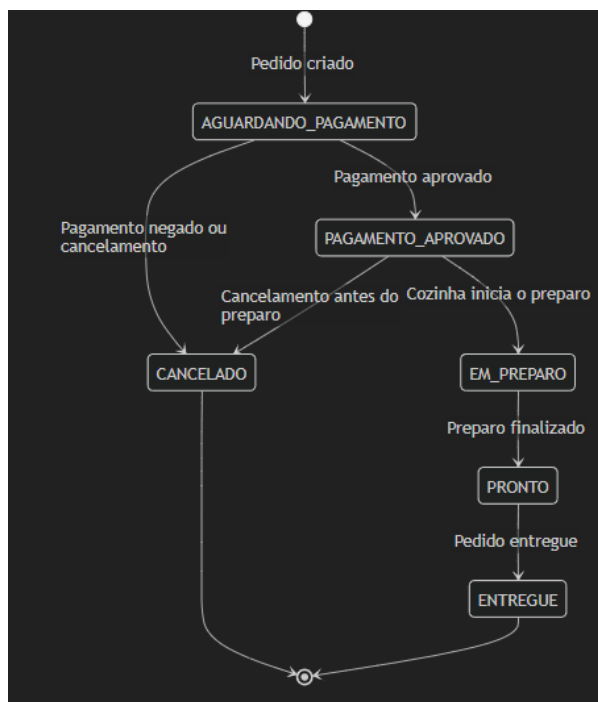
Significa que a cozinha terminou o preparo e o pedido está disponível para entrega ou retirada.

ENTREGUE

Indica que o pedido foi entregue ao cliente. Esse é um status final. Após a entrega, o sistema poderá gerar pontos de fidelidade, desde que o cliente tenha consentimento LGPD ativo.

CANCELADO

Indica que o pedido não continuará. O cancelamento poderá acontecer por pagamento negado, solicitação do cliente ou ação autorizada antes do início do preparo.



Regras

Todo pedido começa em AGUARDANDO_PAGAMENTO. Não existe um status de pedido chamado PAGAMENTO_NEGADO. Quando o pagamento é negado, o pagamento fica como RECUSADO e o pedido muda para CANCELADO. O pedido só pode entrar em EM_PREPARO depois da aprovação do pagamento. O pedido não pode mudar diretamente de AGUARDANDO_PAGAMENTO para PRONTO;

EM_PREPARO somente pode avançar para PRONTO. PRONTO somente pode avançar para ENTREGUE. ENTREGUE e CANCELADO são status finais. O cancelamento comum não será permitido depois que o pedido entrar em EM_PREPARO. Toda alteração deve ser registrada em HISTORICO_STATUS_PEDIDO;

Mudanças sensíveis também devem gerar um registro em LOG_AUDITORIA. Os pontos de fidelidade somente são creditados após ENTREGUE e com consentimento ativo.

Endpoint: mudança de status

PATCH /pedidos/{id}/status

Ex:

```
{
  "status": "EM_PREPARO",
  "observacao": "Preparação iniciada pela cozinha."
}
```

Ou:

POST /pedidos/{id}/cancelamento

Se alguém tentar realizar uma mudança que não seja permitida, a API deverá retornar 409 Conflict.

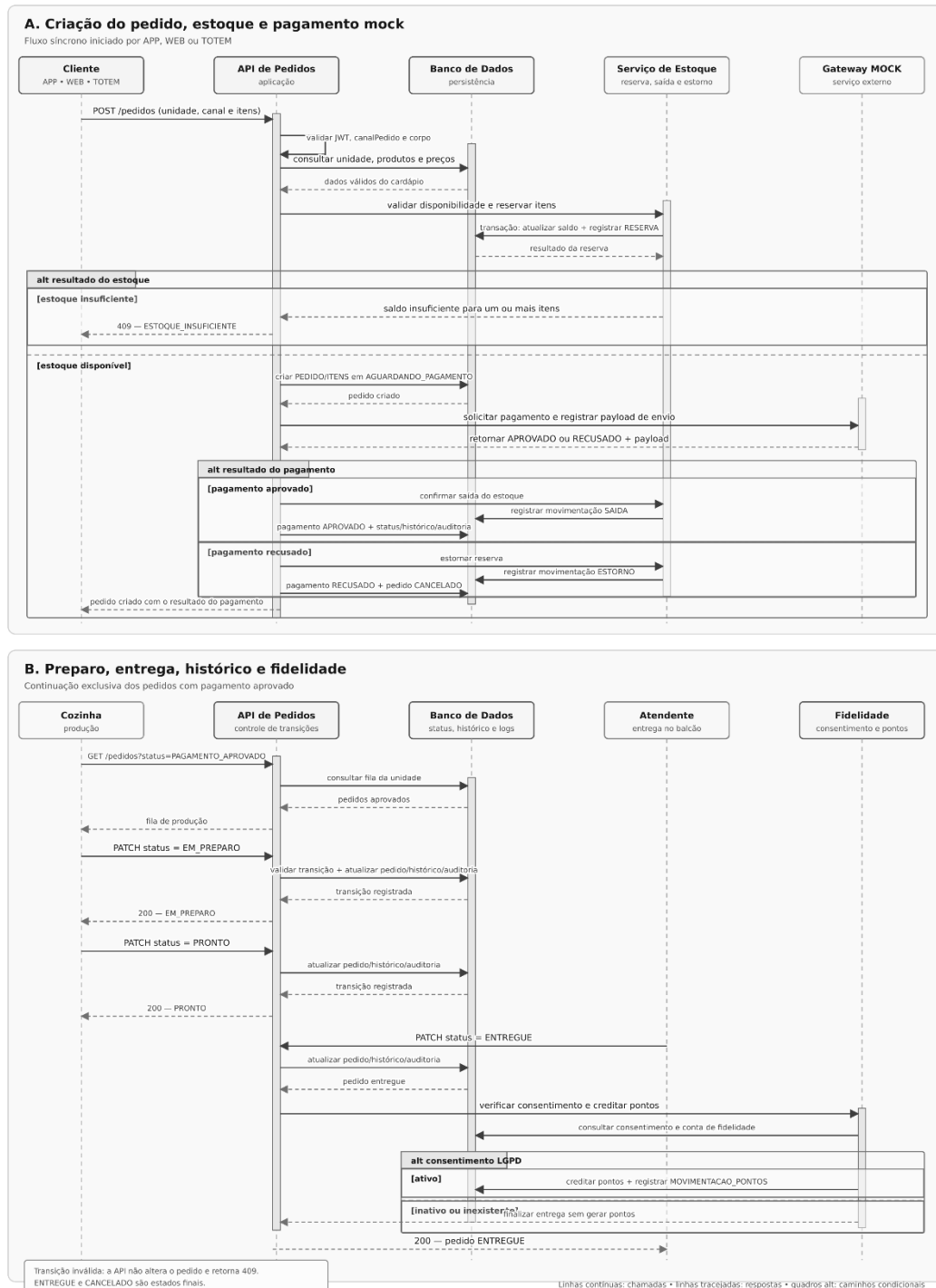
Exemplo de erro:

```
{
  "status": 409,
  "codigo": "TRANSICAO_STATUS_INVALIDA",
  "mensagem": "O pedido não pode mudar de AGUARDANDO_PAGAMENTO para PRONTO.",
  "timestamp": "2026-08-06T10:35:00Z"
}
```

12. Diagrama de sequência do fluxo crítico

O diagrama de sequência apresenta a ordem das interações realizadas durante o fluxo crítico da API. O processo começa com a criação do pedido, passa pela validação e reserva do estoque, pelo pagamento mock e pelas atualizações de status, terminando com a entrega e a possível geração de pontos de fidelidade.

Figura 5 – Diagrama de sequência



Fonte: Autoria própria (2026).

- <https://imgur.com/a/z9ExOpv>
- https://drive.google.com/drive/folders/1pHfzpLul4w-30X2Rj0FdjYI-G4RTqZ99?usp=drive_link

13. Definição da arquitetura e separação em camadas

Para o desenvolvimento da API foi escolhida uma arquitetura em camadas, baseada de forma simplificada nos conceitos da **Clean Architecture**. Essa escolha permite separar melhor as regras de negócio dos detalhes técnicos, como banco de dados, autenticação e integração com pagamento.

A solução será dividida em quatro camadas principais: **Domínio, Aplicação, Infraestrutura e API**. Cada camada possui uma responsabilidade específica e se comunica com as outras de forma organizada.

Camada de Domínio

A camada de Domínio representa a parte central do sistema. Nela ficam as entidades, regras de negócio, validações e comportamentos que não devem depender do banco de dados ou do framework utilizado.

As principais entidades dessa camada são:

- Usuário;
- Unidade;
- Produto;
- Estoque;
- Movimentação de estoque;
- Pedido;
- Item do pedido;
- Pagamento;
- Promoção;
- Consentimento LGPD;
- Conta de fidelidade;
- Movimentação de pontos.

Regras

- APP, WEB ou TOTEM;
- O produto precisa estar disponível na unidade escolhida;
- Não pode ser realizada uma venda sem estoque suficiente;
- Um pedido só pode ir para preparo depois do pagamento ser aprovado;
- Um pedido entregue ou cancelado não pode voltar para um status anterior;

- Pontos de fidelidade só são adicionados após a entrega do pedido;
- O cliente deve possuir consentimento ativo para participar do programa de fidelidade;
- A senha do usuário nunca deve ser armazenada em texto normal.

Essa camada não conhece diretamente detalhes do banco, rotas HTTP ou o funcionamento do pagamento mock.

Camada de Aplicação

Contém os casos de uso do sistema. Ela é responsável por coordenar as entidades do domínio e os serviços necessários para concluir cada operação.

Alguns casos de uso previstos são:

- Cadastrar usuário;
- Autenticar usuário;
- Renovar token de acesso;
- Cadastrar unidade;
- Consultar o cardápio de uma unidade;
- Registrar entrada ou ajuste de estoque;
- Criar pedido;
- Cancelar pedido;
- Atualizar o status do pedido;
- Solicitar pagamento;
- Aplicar promoção;
- Adicionar ou resgatar pontos;
- Registrar consentimento LGPD.

A camada de Aplicação utiliza interfaces de repositórios e serviços. Assim, ela informa o que precisa ser feito, mas não precisa saber exatamente como os dados vão ser salvos.

Camada de Infraestrutura

A camada de Infraestrutura reúne os detalhes técnicos usados pela aplicação. Nela ficarão as implementações dos repositórios, acesso ao banco de dados, migrations, seed e integrações externas.

Essa camada será responsável por:

- Implementar os repositórios do banco de dados;
- Configurar o ORM;
- Executar migrations;
- Inserir dados iniciais através de seed;
- Implementar o serviço de pagamento mock;
- Gerar e validar tokens JWT;

- Gerar o hash das senhas;
- Registrar logs e auditorias;
- Configurar variáveis de ambiente.

O pagamento será apenas uma simulação. Mesmo assim, representa uma integração externa, recebendo os dados da solicitação e retornando situações como pagamento aprovado ou recusado.

Os repositórios também garantem que operações importantes sejam realizadas por meio de transações. Na criação do pedido, por exemplo, a reserva de estoque e o registro do pedido precisam acontecer de forma consistente, evitando que apenas uma parte do processo seja salva.

Essa camada será responsável por:

- Implementar os repositórios do banco de dados;
- Configurar o ORM;
- Executar migrations;
- Inserir dados iniciais através de seed;
- Implementar o serviço de pagamento mock;
- Gerar e validar tokens JWT;
- Gerar o hash das senhas;
- Registrar logs e auditorias;
- Configurar variáveis de ambiente.

O pagamento será apenas uma simulação. Mesmo assim, ele deverá representar uma integração externa, recebendo os dados da solicitação e retornando situações como pagamento aprovado ou recusado.

Os repositórios também devem garantir que operações importantes sejam realizadas por meio de transações. Na criação do pedido, por exemplo, a reserva de estoque e o registro do pedido precisam acontecer de forma consistente, evitando que apenas uma parte do processo seja salva.

Camada de API

A camada de API será o ponto de entrada para os clientes externos, como aplicativo, site e totem. Nessa camada estarão as rotas, controllers, autenticação, autorização e os contratos de request e response.

Suas principais responsabilidades são:

- Receber as requisições HTTP;
- Validar os dados básicos enviados;
- Verificar o token JWT;

- Verificar o perfil e as permissões do usuário;
- Acionar o caso de uso correspondente;
- Converter os resultados em respostas HTTP;
- Retornar erros seguindo um padrão;
- Disponibilizar a documentação Swagger/OpenAPI.

Os controllers possuem pouca regra de negócio. Sua função será receber a requisição, chamar o caso de uso e devolver a resposta correta.

Fluxo entre as camadas

Quando uma requisição chegar na API, ela passará primeiro pela autenticação e pelas validações iniciais. Depois, o controller encaminhará a solicitação para o caso de uso da camada de Aplicação.

O caso de uso utiliza as regras definidas no Domínio e acessa os recursos técnicos por meio de interfaces. A Infraestrutura implementa essas interfaces e faz o acesso ao banco ou a serviços externos.

O fluxo geral será:

- O cliente envia uma requisição para um endpoint;
- A API verifica autenticação, permissão e dados obrigatórios;
- O controller chama o caso de uso;
- O caso de uso executa as regras de negócio;
- Os repositórios acessam o banco de dados;
- O resultado retorna para o controller;
- A API envia uma resposta padronizada ao cliente.

As dependências devem apontar para as camadas mais internas. Assim, o Domínio não depende da API, do banco ou do pagamento mock. Isso permite trocar alguma tecnologia futuramente sem precisar alterar todas as regras de negócio.

Organização inicial das pastas em ingles

src

- domain
 - entities
 - enums
 - exceptions
 - repositories
 - services
- application
 - usecases
 - dto

- interfaces
- infrastructure
 - database
 - migrations
 - seed
 - repositories
 - payment
 - security
 - audit
- api
 - controllers
 - routes
 - middlewares
 - validators
 - documentation
- main
 - config
 - server

Segurança e recursos compartilhados

Algumas funções serão usadas por várias partes do sistema, como autenticação, validação, logs, tratamento de erros e auditoria. Esses recursos serão integrados às camadas sem colocar todas as responsabilidades dentro dos controllers.

- Senhas armazenadas com hash;
- Autenticação com JWT;
- Refresh token com possibilidade de revogação;
- Controle de acesso por perfis;
- Validação dos dados de entrada;
- Logs para erros e operações importantes;
- Auditoria de ações sensíveis;
- Respostas sem exposição de senha, token ou informações internas;
- Registro do consentimento relacionado ao programa de fidelidade.

14. Definição dos recursos da API

A API está organizada em recursos que representam as principais partes do sistema da rede de lanchonetes. Cada recurso reúne informações e operações relacionadas a uma função específica, como autenticação, unidades, produtos, estoque e pedidos.

Como o projeto é voltado somente ao back-end, as telas do aplicativo, site e totem não fazem parte da implementação. Esses canais apenas representam as possíveis origens das requisições, sendo identificados nos pedidos pelo campo obrigatório canalPedido, que aceita os valores APP, WEB ou TOTEM.

Autenticação

O recurso de autenticação é responsável pelo acesso seguro à API. Ele permite realizar cadastro, login, renovação do token e encerramento da sessão.

A senha é armazenada com hash e, depois do login, o usuário recebe um token JWT. O refresh token é controlado por meio das sessões registradas no banco de dados.

Usuários

O recurso de usuários mantém os dados necessários para identificação e controle de acesso. Cada usuário possui um perfil que determina suas permissões dentro do sistema.

Os perfis definidos são:

- Cliente;
- Atendente;
- Cozinha;
- Gerente;
- Administrador.

Os usuários internos podem estar vinculados a uma unidade da rede. Dados sensíveis, como senha e tokens, não aparecem nas respostas da API.

Unidades

O recurso de unidades representa os estabelecimentos da rede. Ele permite cadastrar, consultar e alterar informações das unidades, além de identificar a relação entre a matriz e suas unidades vinculadas.

Cada unidade possui seu próprio cardápio, preços e controle de estoque. Dessa forma, a disponibilidade de um produto pode ser diferente de uma unidade para outra.

Produtos e cardápio

O recurso de produtos armazena as informações gerais dos itens vendidos, como nome, descrição e situação ativa ou inativa.

O cardápio é organizado de acordo com cada unidade. A ligação entre produto e unidade define:

- Preço praticado;
- Disponibilidade para venda;
- Participação do produto no cardápio daquela unidade.

Essa separação permite que o mesmo produto possua preços ou disponibilidades diferentes em cada estabelecimento.

Estoque

O recurso de estoque controla a quantidade disponível de cada produto por unidade. Ele permite consultar o saldo e registrar movimentações de entrada, reserva, saída, estorno e ajuste.

Durante a criação de um pedido, a API verifica se existe quantidade suficiente dos produtos. Caso o estoque seja insuficiente, o pedido não continua e o sistema retorna um erro padronizado.

As movimentações ficam registradas para garantir rastreabilidade e evitar alterações no saldo sem identificação da origem.

Pedidos

O recurso de pedidos é o núcleo principal da API. Ele é responsável pela criação, consulta, cancelamento e atualização de status dos pedidos.

Todo pedido possui:

- Cliente;
- Unidade;
- Um ou mais itens;
- Quantidades;
- Valores;
- Forma de pagamento;
- Status;
- Canal do pedido.

O campo canalPedido é obrigatório e aceita somente APP, WEB ou TOTEM. A consulta dos pedidos também permite utilizar o canal e o status como filtros.

Os itens, o histórico de status e os valores permanecem relacionados ao pedido, não sendo tratados como recursos completamente independentes.

Pagamentos

O recurso de pagamentos representa a solicitação enviada para um serviço externo. Neste projeto, a integração utiliza um gateway mock, sem qualquer comunicação com instituições financeiras reais.

O mock retorna pagamento aprovado ou recusado. A solicitação, o status e os payloads de envio e retorno ficam registrados no banco, demonstrando o fluxo da integração.

Quando o pagamento é aprovado, o pedido pode seguir para o preparo. Quando é recusado, o pedido é cancelado e a reserva do estoque é estornada.

Fidelização e consentimento

O recurso de fidelização controla o saldo e as movimentações de pontos do cliente. A implementação é simples e permite adicionar pontos depois da entrega do pedido, além de realizar o resgate conforme uma regra definida.

A participação no programa depende do consentimento ativo do cliente. O sistema registra a finalidade, a data e a situação desse consentimento.

Caso o cliente não possua consentimento ativo, o pedido continua normalmente, mas os pontos de fidelidade não são adicionados.

Promoções

O recurso de promoções aparece de forma simplificada no modelo de dados. Ele armazena informações como nome, período de validade, tipo de desconto e situação da campanha.

A aplicação de campanhas mais avançadas fica apenas como proposta conceitual, já que o requisito permite que essa parte seja apresentada por meio de regras e documentação. Combinações complexas de cupons ou descontos não fazem parte da implementação.

Auditoria

O recurso de auditoria registra ações importantes, como login, alteração de estoque, criação ou cancelamento de pedido, mudança de status, resposta do pagamento e movimentação de pontos.

Esses registros servem para rastreabilidade e segurança. A consulta completa da auditoria fica restrita ao perfil de administrador.

Recursos internos e auxiliares

Algumas informações fazem parte do banco e da lógica do sistema, mas não possuem operações públicas próprias. Entre elas estão:

- Sessões de refresh token;
- Itens do pedido;

- Histórico de status;
- Movimentações de estoque;
- Movimentações de pontos;
- Logs de auditoria.

Esses dados são criados e consultados por meio dos recursos principais aos quais pertencem.

Dessa forma, os recursos da API permanecem ligados ao DER e ao fluxo crítico definido anteriormente. A criação do pedido utiliza o usuário, a unidade, o cardápio e o estoque. Em seguida, o sistema solicita o pagamento mock, atualiza o status e, quando aplicável, adiciona os pontos de fidelidade e registra a auditoria.

15. Endpoints

Auth - Cadastro

Finalidade: Cadastrar um cliente.

Método/Rota: POST /auth/cadastro

Auth/Permissão: Pública.

Parâmetros: Nenhum.

Request: (JSON)

```
{
  nome: "João Silva",
  email: "joao@email.com",
  senha: "Senha@123"
}
```

Response 200/201 (JSON):

```
{
  id: "uuid-usuario",
  nome: "João Silva",
  email: "joao@email.com",
  perfil: "CLIENTE"
}
```

Status codes: 201, 409, 422

Erros (padrão):

```
{
  timestamp: "2026-08-08T12:00:00.000Z",
  status: 409,
  code: "409 – EMAIL_JA_CADASTRADO",
  message: "O e-mail informado já está cadastrado.",
  path: "/path",
  details: [
    {
      field: "campo",
      issue: "O e-mail informado já está cadastrado."
    }
  ],
  requestId: "uuid-da-requisicao"
}
```

2. Login

Auth - Login

Finalidade: Autenticar o usuário e retornar os tokens.

Método/Rota: POST /auth/login

Auth/Permissão: Pública.

Parâmetros: Nenhum.

Request: (JSON)

```
{
  email: "joao@email.com",
  senha: "Senha@123"
}
```

Response 200/201 (JSON):

```
{
  accessToken: "token-jwt",
  refreshToken: "refresh-token",
  tokenType: "Bearer",
  expiresIn: 900,
  usuario: {
    id: "uuid-usuario",
    nome: "João Silva",
    perfil: "CLIENTE" }
}
```

Status codes: 200, 401, 422

Erros (padrão):

```
{
  timestamp: "2026-08-08T12:00:00.000Z",
  status: 401,
  code: "401 - CREDENCIAIS_INVALIDAS",
  message: "E-mail ou senha inválidos.",
  path: "/auth/login",
  details: []
}
```

3. Renovar token

Auth - Renovar token

Finalidade: Gerar um novo token de acesso.

Método/Rota: POST /auth/refresh

Auth/Permissão: Refresh token válido.

Parâmetros: Nenhum.

Request: (JSON)

```
{
  refreshToken: "refresh-token"
}
```

Response 200/201 (JSON):

```
{
  accessToken: "novo-token-jwt",
  refreshToken: "novo-refresh-token",
  tokenType: "Bearer",
  expiresIn: 900
}
```

Status codes: 200, 401

Erros (padrão):

```
{
  timestamp: "2026-08-08T12:00:00.000Z",
  status: 401,
  code: "401 - REFRESH_TOKEN_INVALIDO",
  message: "O refresh token é inválido, expirado ou foi revogado.",
  path: "/auth/refresh",
  details: [
    {
      field: "refreshToken",
      issue: "O token é inválido, expirado ou foi revogado."
    }
  ]
}
```

4. Logout

Auth - Logout

Finalidade: Encerrar a sessão do usuário.

Método/Rota: POST /auth/logout

Auth/Permissão: JWT – todos os perfis.

Parâmetros: Nenhum.

Request: (JSON)

```
{
  refreshToken: "refresh-token"
}
```

Response 204

Status codes: 204, 401

Erros (padrão):

```
{
  timestamp: "2026-08-08T12:00:00.000Z",
  status: 401,
  code: "401 – NAO_AUTENTICADO",
  message: "Token de autenticação ausente ou inválido.",
  path: "/auth/logout",
  details: []
}
```

5 listar unidades

Unidades - Listar Unidades

Finalidade: Consultar as unidades da rede.

Método/Rota: GET /unidades

Auth/Permissão: Pública.

Parâmetros: Query opcional ativo.

Request: (JSON)

Não possui corpo JSON.

Exemplo: GET /unidades?ativo=true

Response 200

```
{
  items: [
    {
      id: "uuid-unidade",
      nome: "Unidade Centro",
      tipo: "FRANQUIA",
      ativo: true
    }
  ],
  total: 1
}
```

Status codes: 200, 422

Erros (padrão):

```
{
  timestamp: "2026-08-08T12:00:00.000Z",
  status: 402,
  code: "422 - PARAMETRO_INVALIDO",
  message: "O valor informado no filtro ativo é inválido.",
  path: "/unidades",
  details: [
    {
      field: "ativo",
      issue: "O filtro deve receber true ou false."
    }
  ]
}
```

6. Cadastrar unidade

Unidades - Cadastrar unidade

Finalidade: Cadastrar uma unidade da rede.

Método/Rota: POST /unidades

Auth/Permissão: JWT – ADMIN.

Parâmetros: Nenhum.

Request: (JSON)

```
{
  nome: "Unidade Centro",
  tipo: "FRANQUIA",
  unidadeMatrizId: "uuid-matriz",
  endereco: {
    logradouro: "Rua Principal",
    numero: "100",
    bairro: "Centro",
    cidade: "Brasília",
    uf: "DF",
    cep: "70000-000"
  }
}
```

Response 201

```
{
  id: "uuid-unidade",
  nome: "Unidade Centro",
  tipo: "FRANQUIA",
  unidadeMatrizId: "uuid-matriz",
  ativo: true
}
```

Status codes: 201, 401, 403, 404, 422

Erros (padrão):

```
{
  timestamp: "2026-08-08T12:00:00.000Z",
  status: 403,
  code: "403 – ACESSO_NEGADO",
  message: "O usuário não possui permissão para cadastrar unidades.",
  path: "/unidades",
  details: []
}
```

7. Atualizar unidade

Unidades - Atualizar unidade Finalidade: Alterar os dados de uma unidade. Método/Rota: PATCH /unidades/{id} Auth/Permissão: JWT – ADMIN ou GERENTE. Parâmetros: Path id, identificador da unidade. Request: (JSON) <pre>{ nome: "Unidade Centro Atualizada", ativo: true }</pre> Response 200 <pre>{ id: "uuid-unidade", nome: "Unidade Centro Atualizada", tipo: "FRANQUIA", ativo: true }</pre> Status codes: 200, 401, 403, 404, 422 Erros (padrão): <pre>{ timestamp: "2026-08-08T12:00:00.000Z", status: 404, code: "404 – UNIDADE_NAO_ENCONTRADA", message: "A unidade informada não foi encontrada.", path: "/unidades/uuid-unidade", details: [{ field: "id", issue: "Não existe uma unidade com o identificador informado." }] }</pre>
--

8. Cadastrar produto

Produtos - Cadastrar produto Finalidade: Cadastrar um produto no catálogo geral. Método/Rota: POST /produtos Auth/Permissão: JWT – ADMIN ou GERENTE. Parâmetros: Nenhum. Request: (JSON) <pre>{ nome: "X-Burguer", descricao: "Hambúrguer, queijo e salada", categoria: "LANCHE", ativo: true }</pre>
Response 201 <pre>{ id: "uuid-produto", nome: "X-Burguer", descricao: "Hambúrguer, queijo e salada", categoria: "LANCHE", ativo: true }</pre>
Status codes: 201, 401, 403, 409, 422 Erros (padrão): <pre>path: "/produtos", details: [{ field: "nome", issue: "Já existe um produto com esse nome." }]</pre>

9. Consultar cardápio de cada unidade

Produtos e Cardapio - Consultar cardápio Finalidade: Listar os produtos disponíveis em uma unidade. Método/Rota: GET /unidades/{unidadeld}/cardapio Auth/Permissão: Pública Parâmetros: Path unidadeld; query opcional categoria. Request: (JSON)
Response 200 <pre>{ unidadeld: "uuid-unidade", items: [{ produtoid: "uuid-produto", nome: "X-Burguer", categoria: "LANCHE", preco: 24.90, disponivel: true }] }</pre>
Status codes: 200, 404, 422 Erros (padrão): <pre>{ timestamp: "2026-08-08T12:00:00.000Z", status: 404, code: "404 – UNIDADE_NAO_ENCONTRADA", message: "A unidade informada não foi encontrada.", path: "/unidades/uuid-unidade/cardapio", details: [{ field: "unidadeld", issue: "Não existe uma unidade com o identificador informado." }] }</pre>

10. Configurar produto no cardápio

Cardápio - Configurar produto no cardápio

Finalidade: Incluir ou atualizar um produto no cardápio da unidade.

Método/Rota: PUT /unidades/{unidadeld}/cardapio/{produtold}

Auth/Permissão: JWT – ADMIN ou GERENTE.

Parâmetros: Path unidadeld e produtold.

Request: (JSON)

```
{
  preco: 24.90,
  disponivel: true
}
```

Response 200

```
{
  unidadeld: "uuid-unidade",
  produtold: "uuid-produto",
  preco: 24.90,
  disponivel: true
}
```

Status codes: 200, 401, 403, 404, 422

Erros (padrão):

```
{
  timestamp: "2026-08-08T12:00:00.000Z",
  status: 404,
  code: "404 – PRODUTO_NAO_ENCONTRADO",
  message: "O produto informado não foi encontrado.",
  path: "/unidades/uuid-unidade/cardapio/uuid-produto",
  details: [
    {
      field: "produtold",
      issue: "Não existe um produto com o identificador informado."
    }
  ]
}
```

11. Consultar estoque por unidade

Estoque - Consultar estoque por unidade

Finalidade: Consultar as quantidades dos produtos em uma unidade.

Método/Rota: GET /estoque

Auth/Permissão: JWT – ADMIN, GERENTE, ATENDENTE ou COZINHA.

Parâmetros: Query obrigatória unidadeId; query opcional produtoId.

Request: (JSON)

Exemplo: GET /estoque?unidadeId=uuid-unidade

Response 200

```
{
  items: [
    {
      produtoId: "uuid-produto",
      nome: "X-Burguer",
      quantidadeDisponivel: 30,
      quantidadeReservada: 2
    }
  ]
}
```

Status codes: 200, 401, 403, 404, 422

Erros (padrão):

```
{
  timestamp: "2026-08-08T12:00:00.000Z",
  status: 404,
  code: "404 – UNIDADE_NAO_ENCONTRADA",
  message: "A unidade informada não foi encontrada.",
  path: "/estoque",
  details: [
    {
      field: "unidadeId",
      issue: "Não existe uma unidade com o identificador informado."
    }
  ]
}
```

12. Movimentar estoque

Estoque - Registrar movimentação

Finalidade: Registrar entrada ou ajuste manual de estoque.

Método/Rota: POST /estoque/movimentacoes

Auth/Permissão: JWT – ADMIN ou GERENTE.

Parâmetros: Nenhum.

Request: (JSON)

```
{
  unidadeld: "uuid-unidade",
  produtoid: "uuid-produto",
  tipo: "ENTRADA",
  quantidade: 20,
  motivo: "Reposição semanal"
}
```

Response 201

```
{
  id: "uuid-movimentacao",
  tipo: "ENTRADA",
  quantidade: 20,
  saldoAtual: 50,
  dataMovimentacao: "2026-08-08T12:00:00.000Z"
}
```

Status codes: 201, 401, 403, 404, 409, 422

Erros (padrão):

```
{
  timestamp: "2026-08-08T12:00:00.000Z",
  status: 422,
  code: "422 – MOVIMENTACAO_INVALIDA",
  message: "A quantidade da movimentação deve ser maior que zero.",
  path: "/estoque/movimentacoes",
  details: [
    {
      field: "quantidade",
      issue: "A quantidade deve ser maior que zero."
    }
  ]
}
```

13. Criar pedido

Pedidos - Criar pedido

Finalidade: Criar um pedido, reservar o estoque e solicitar o pagamento mock.

Método/Rota: POST /pedidos

Auth/Permissão: JWT – CLIENTE ou ATENDENTE.

Parâmetros: Nenhum.

Request (JSON)

```
{
  canalPedido: "TOTEM",
  clientId: "uuid-cliente",
  unidadeId: "uuid-unidade",
  itens: [
    {
      produtoId: "uuid-produto",
      quantidade: 2
    }
  ],
  formaPagamento: "MOCK",
  tokenPagamento: "mock_aprovado"
}
```

Response 201

```
{
  id: "uuid-pedido",
  canalPedido: "TOTEM",
  status: "PAGAMENTO_APROVADO",
  valorTotal: 49.80,
  pagamento: {
    provedor: "MOCK",
    status: "APROVADO"
  }
}
```

Status codes: 201, 401, 403, 404, 405, 422

Erros (padrão):

```
{
  timestamp: "2026-08-08T12:00:00.000Z",
  status: 409,
  code: "409 – ESTOQUE_INSUFICIENTE",
  message: "Não há estoque suficiente para concluir o pedido.",

  path: "/pedidos",
  details: { details: itens[0].quantidade
    {
      field: "itens[0].quantidade",
      issue: "A quantidade solicitada é maior que o estoque disponível."
    }
  }
  path: "/pedidos",
  details: {
    {
      field: "canalPedido",
      issue: "O canal deve ser APP, WEB ou TOTEM."
    }
  }
}
```

14. Listar pedidos

Pedidos - Listar pedidos	
Finalidade: Consultar pedidos utilizando filtros.	
Método/Rota: GET /pedidos	
Auth/Permissão: JWT – todos os perfis, conforme o nível de acesso.	
Parâmetros: Query canalPedido, status, unidadeId, page e limit.	
Request: (JSON)	
N/A	
GET /pedidos?canalPedido=APP&status=AGUARDANDO_PAGAMENTO	
Response 200	
<pre>{ items: [{ id: "uuid-pedido", canalPedido: "APP", status: "AGUARDANDO_PAGAMENTO", valorTotal: 49.80 }], page: 1, limit: 20, total: 1 }</pre>	
Status codes: 200, 401, 403, 422	
Erros (padrão):	
<pre>{ timestamp: "2026-08-08T12:00:00.000Z", status: 422, code: "422 – FILTRO_INVALIDO", message: "O canal informado no filtro é inválido.", path: "/pedidos", details: [{ field: "canalPedido", issue: "O filtro deve ser APP, WEB ou TOTEM." }] }</pre>	

15. Atualizar status do pedido

Pedidos - Atualizar status

Finalidade: Atualizar a etapa operacional do pedido.

Método/Rota: PATCH /pedidos/{id}/status

Auth/Permissão: JWT – COZINHA, ATENDENTE, GERENTE ou ADMIN.

Parâmetros: Path id, identificador do pedido.

Request: (JSON)

```
{
  status: "EM_PREPARO"
}
```

Response 200

```
{
  id: "uuid-pedido",
  statusAnterior: "PAGAMENTO_APROVADO",
  statusAtual: "EM_PREPARO",
  atualizadoEm: "2026-08-08T12:00:00.000Z"
}
```

Status codes: 200, 401, 403, 404, 409, 422

Erros (padrão):

```
{
  timestamp: "2026-08-08T12:00:00.000Z",
  status: 409,
  code: "409 – TRANSICAO_STATUS_INVALIDA",
  message: "A transição solicitada não é permitida para o status atual.",
  path: "/pedidos/uuid-pedido/status",
  details: [
    {
      field: "status",
      issue: "Não é permitido passar de AGUARDANDO_PAGAMENTO diretamente para PRONTO."
    }
  ]
}
```

16. Cancelar pedido

Pedidos - Cancelar pedido

Finalidade: Cancelar um pedido permitido e estornar o estoque reservado.

Método/Rota: POST /pedidos/{id}/cancelamento

Auth/Permissão: JWT – CLIENTE, ATENDENTE, GERENTE ou ADMIN.

Parâmetros: Path id, identificador do pedido.

Request: (JSON)

```
{
  motivo: "Cliente desistiu do pedido"
}
```

Response 200

```
{
  id: "uuid-pedido",
  status: "CANCELADO",
  motivo: "Cliente desistiu do pedido",
  estoqueEstornado: true
}
```

Status codes: 200, 401, 403, 404, 409, 422

Erros (padrão):

```
{
  timestamp: "2026-08-08T12:00:00.000Z",
  status: 409,
  code: "409 – CANCELAMENTO_NAO_PERMITIDO",
  message: "O pedido não pode ser cancelado no status atual.",
  path: "/pedidos/uuid-pedido/cancelamento",
  details: [
    {
      field: "pedido.status",
      issue: "O pedido está EM_PREPARO e não permite cancelamento comum."
    }
  ]
}
```

17. Consultar pagamento

Pagamentos - Consultar pagamento do pedido

Finalidade: Consultar o registro e o retorno do pagamento mock.

Método/Rota: GET /pedidos/{pedidoId}/pagamento

Auth/Permissão: JWT – usuário autorizado a consultar o pedido.

Parâmetros: Path pedidoId.

Request: (JSON)

Response 200

```
{
  id: "uuid-pagamento",
  pedidoId: "uuid-pedido",
  provedor: "MOCK",
  status: "APROVADO",
  payloadEnvio: {
    valor: 49.80,
    formaPagamento: "MOCK"
  },
  payloadRetorno: {
    codigo: "00",
    mensagem: "Pagamento aprovado"
  }
}
```

Status codes: 200, 401, 403, 404

Erros (padrão):

```
{
  timestamp: "2026-08-08T12:00:00.000Z",
  status: 404,
  code: "404 – PAGAMENTO_NAO_ENCONTRADO",
  message: "Não foi encontrado um pagamento para o pedido informado.",
  path: "/pedidos/uuid-pedido/pagamento",
  details: [
    {
      field: "pedidoId",
      issue: "Não existe pagamento registrado para o pedido informado."
    }
  ]
}
```

18. Consultar saldo

Consultar saldo

Fidelidade - Consultar saldo

Finalidade: Consultar o saldo de pontos do cliente autenticado.

Método/Rota: GET /fidelidade/saldo

Auth/Permissão: JWT – CLIENTE.

Parâmetros: Nenhum.

Request: (JSON)

Response 200

```
{
  clientId: "uuid-cliente",
  saldoPontos: 250,
  consentimentoAtivo: true
}
```

Status codes: 200, 401, 403

Erros (padrão):

```
{
  timestamp: "2026-08-08T12:00:00.000Z",
  status: 403,
  code: "403 – PERFIL_NAO_AUTORIZADO",
  message: "Somente clientes podem consultar o saldo de fidelidade.",
  path: "/fidelidade/saldo",
  details: []
}
```

19. Resgatar pontos

Fidelidade - Resgatar pontos

Finalidade: Registrar o resgate simples de pontos do cliente.

Método/Rota: POST /fidelidade/resgates

Auth/Permissão: JWT – CLIENTE.

Parâmetros: Nenhum.

Request: (JSON)

```
{  
  pontos: 100  
}
```

Response 200

```
{  
  id: "uuid-movimentacao",  
  pontosResgatados: 100,  
  valorConvertido: 5.00,  
  saldoAtual: 150  
}
```

Status codes: 201, 401, 403, 409, 422

Erros (padrão):

```
{  
  timestamp: "2026-08-08T12:00:00.000Z",  
  status: 409,  
  code: "409 – SALDO_PONTOS_INSUFICIENTE",  
  message: "O cliente não possui pontos suficientes para o resgate.",  
  path: "/fidelidade/resgates",  
  details: [  
    {  
      field: "pontos",  
      issue: "A quantidade solicitada é maior que o saldo disponível."  
    }  
  ]  
}
```

20. Registrar consentimento

Consentimentos - Registrar consentimento Finalidade: Registrar a concessão ou revogação do consentimento. Método/Rota: POST /consentimentos Auth/Permissão: JWT – CLIENTE. Parâmetros: Nenhum. Request: (JSON) <pre>{ finalidade: "PROGRAMA_FIDELIDADE", aceito: true, versaoTermo: "1.0" }</pre> Response 201 <pre>{ id: "uuid-consentimento", finalidade: "PROGRAMA_FIDELIDADE", aceito: true, versaoTermo: "1.0", registradoEm: "2026-08-08T12:00:00.000Z" }</pre> Status codes: 201, 401, 403, 422 Erros (padrão): <pre>{ timestamp: "2026-08-08T12:00:00.000Z", status: 422, code: "CONSENTIMENTO_INVALIDO", message: "Os dados do consentimento são inválidos.", path: "/consentimentos", details: [{ field: "finalidade", issue: "A finalidade informada não é aceita pelo sistema." }], requestId: "uuid-da-requisicao" }</pre>
--

16. Definição dos contratos de request e response

Os contratos definem quais dados a API recebe e o que ela devolve em cada endpoint. As informações são enviadas e retornadas no formato JSON, com exceção de respostas sem conteúdo, como acontece no logout.

Os requests possuem somente os campos necessários para realizar cada operação. Campos obrigatórios que não forem enviados, ou estiverem em um formato inválido, geram erro de validação.

Alguns padrões utilizados nos contratos são:

- Identificadores no formato UUID;
- Datas e horários no padrão ISO 8601;
- Valores monetários com duas casas decimais;
- Enums escritos em letras maiúsculas;
- Quantidades representadas por números inteiros;
- Campos booleanos representados por true ou false.

Um exemplo é o contrato de criação do pedido:

```
{
  "canalPedido": "TOTEM",
  "clienteId": "uuid-cliente",
  "unidadeId": "uuid-unidade",
  "itens": [
    {
      "produtoId": "uuid-produto",
      "quantidade": 2
    }
  ],
  "formaPagamento": "MOCK",
  "tokenPagamento": "mock_aprovado"
}
```

A resposta apresenta o pedido criado e o resultado do pagamento mock:

```
{
  "id": "uuid-pedido",
  "canalPedido": "TOTEM",
  "status": "PAGAMENTO_APROVADO",
  "valorTotal": 49.80,
  "pagamento": {
    "provedor": "MOCK",
```

```
"status": "APROVADO"
}
}
```

O campo canalPedido é obrigatório e aceita somente APP, WEB ou TOTEM. Quando um cliente cria o próprio pedido, o clientId deve corresponder ao usuário identificado no token. O atendente pode informar o cliente relacionado ao atendimento.

As respostas não apresentam senha, hash, refresh token armazenado ou detalhes internos do sistema. No caso do pagamento, somente informações necessárias para demonstrar o mock são retornadas.

Os contratos completos de cada endpoint aparecem nas fichas apresentadas anteriormente e também ficam disponíveis no Swagger/OpenAPI.

17. Definição dos parâmetros de rota, consulta e filtros

Os endpoints utilizam parâmetros de rota, query e corpo da requisição. Cada tipo possui uma finalidade diferente.

Os parâmetros de rota identificam um registro específico. Alguns exemplos são:

/unidades/{id}

/unidades/{unidadeId}/cardapio

/pedidos/{id}/status

/pedidos/{pedidoId}/pagamento

Os principais parâmetros de rota são:

Id: Identifica o registro principal da rota

unidadeId: Identifica uma unidade

produtoId: Identifica um produto

pedidoId: Identifica um pedido

Todos esses identificadores utilizam o formato UUID. Quando o valor é inválido ou o registro não existe, a API retorna um erro apropriado.

Os parâmetros de consulta aparecem depois do sinal ? e são usados principalmente para filtros. Por exemplo:

GET /pedidos?canalPedido=APP&status=AGUARDANDO_PAGAMENTO

No estoque, unidadeId é obrigatório, pois o saldo sempre pertence a uma unidade. Nos outros casos, os filtros são opcionais.

A paginação utiliza page e limit. Quando não são informados, a API considera a página 1 e o limite de 20 registros. O limite máximo é de 100 registros por consulta, evitando respostas muito grandes.

Os dados usados para criar ou alterar recursos são enviados no corpo da requisição. Isso acontece no cadastro, login, criação de unidade, movimentação de estoque, criação de pedido, alteração de status e registro de consentimento.

Caso um parâmetro tenha um valor que não existe nos enums ou esteja em um formato inválido, a API retorna o status 422.

18. Definição dos códigos de status HTTP

Os códigos HTTP indicam o resultado da requisição. A API utiliza os mesmos códigos em todos os recursos para não deixar cada endpoint com um comportamento diferente.

Código	Significado	Situação de uso
200	Sucesso	Consultas e atualizações realizadas
201	Criado	Cadastro ou criação de um novo registro
204	Sem conteúdo	Logout realizado sem corpo na resposta
400	Requisição inválida	JSON malformatado ou requisição impossível de interpretar
401	Não autenticado	Token ausente, expirado ou inválido
403	Acesso negado	Usuário autenticado, mas sem permissão
404	Não encontrado	Registro informado não existe
409	Conflito	Regra de negócio impede a operação
422	Dados inválidos	Campo obrigatório ausente ou em formato incorreto
500	Erro interno	Falha inesperada no servidor

O status 409 é utilizado em situações como:

- Estoque insuficiente;
- Transição de status não permitida;
- Cancelamento não permitido;
- Saldo de pontos insuficiente;
- E-mail ou produto já cadastrado.

O status 422 é utilizado quando o problema está nos dados enviados, como canal do pedido inválido, quantidade igual a zero ou campo obrigatório ausente.

Um pagamento recusado pelo mock não representa uma falha HTTP. A API conseguiu enviar e processar a solicitação normalmente. Nesse caso, o pedido é registrado, o pagamento recebe o status RECUSADO e o pedido passa para CANCELADO.

19. Definição do padrão de respostas e erros

As respostas de sucesso seguem um formato simples. Quando o endpoint retorna apenas um registro, o objeto é apresentado diretamente:

```
{
  "id": "uuid-unidade",
  "nome": "Unidade Centro",
  "tipo": "FRANQUIA",
  "ativo": true
}
```

Nas listagens, a resposta utiliza o campo items. Quando existe paginação, também são apresentados page, limit e total:

```
{
  "items": [
    {
      "id": "uuid-pedido",
      "canalPedido": "APP",
      "status": "AGUARDANDO_PAGAMENTO",
      "valorTotal": 49.80
    }
  ],
  "page": 1,
  "limit": 20,
  "total": 1
}
```

As respostas de erro seguem o mesmo padrão em toda a API:

```
{
  "timestamp": "2026-08-08T12:00:00.000Z",
  "status": 422,
  "code": "CANAL_PEDIDO_INVALIDO",
  "message": "O canal informado não é válido.",
  "path": "/pedidos",
  "details": [
    {
      "field": "canalPedido",
      "issue": "O canal deve ser APP, WEB ou TOTEM."
    }
  ],
  "requestId": "uuid-da-requisicao"
}
```

Cada campo possui uma função:

Timestamp	Data e horário do erro
Status	Código HTTP retornado
Code	Código interno e estável do erro
Message	Mensagem fácil de entender
Path	Rota onde o erro aconteceu
Details	Campos que causaram o problema
requestId	Identificador usado para localizar a requisição nos logs

Quando o erro não está relacionado a um campo específico, details retorna uma lista vazia: "details": []

Erros inesperados não exibem consultas SQL, stack trace, senha, token ou configurações internas. A resposta apresenta apenas uma mensagem genérica, enquanto os detalhes técnicos ficam registrados nos logs da aplicação.

Esse padrão também aparece no Swagger, nos testes e na coleção do Postman, mantendo a documentação igual ao funcionamento real da API.

20. Plano de testes da API

O plano de testes foi elaborado para verificar o funcionamento dos principais recursos da API. Os cenários incluem autenticação, autorização por perfil, validação dos dados, criação e consulta de pedidos, controle de estoque, pagamento mock,

atualização de status, fidelidade e auditoria. Foram definidos seis cenários positivos e quatro negativos, permitindo que os testes sejam repetidos pelo avaliador através da coleção do Postman.

Login do administrador

Tipo: Positivo

Endpoint e método: POST /auth/login

Pré-condição: O seed deve ter criado o administrador.

Entrada:

```
{  
  "email": "admin@lanchonete.local",  
  "senha": "Senha123!"  
}
```

Saída esperada: Status 200. A resposta apresenta accessToken e refreshToken, sem mostrar a senha ou seu hash.

Evidência: Requisição T01 - Login de administrador na pasta Auth da coleção Postman.

Consulta pública do cardápio

Tipo: Positivo

Endpoint e método: GET /unidades/{unidadeId}/cardapio

Pré-condição: A unidade e seus produtos devem ter sido criados pelo seed.

Entrada de rota:

unidadeId = 00000000-0000-4000-8000-000000000001

Saída esperada: Status 200. A resposta apresenta os produtos disponíveis, seus preços e o produtoId. A rota não exige token.

Evidência: Requisição T02 - Consultar cardápio na pasta Produtos.

Criação de pedido com pagamento aprovado

Tipo: Positivo

Endpoint e método: POST /pedidos

Pré-condição: Cliente autenticado, produto disponível no cardápio e estoque suficiente.

Entrada:

```
{
  "canalPedido": "TOTEM",
  "clienteId": "00000000-0000-4000-8000-000000000012",
  "unidadeId": "00000000-0000-4000-8000-000000000001",
  "itens": [
    {
      "produtoId": "{{produtoId}}",
      "quantidade": 1
    }
  ],
  "formaPagamento": "MOCK"
}
```

Saída esperada: Status 201. O pedido é criado com canal TOTEM, o pagamento fica APROVADO e o pedido assume o status PAGAMENTO_APROVADO. O estoque também registra a reserva e a saída.

Evidência: Requisição T03 - Criar pedido aprovado na pasta Pedidos.

Filtro de pedidos por canal e status

Tipo: Positivo

Endpoint e método: GET /pedidos

Pré-condição: Administrador autenticado e pedido do cenário T03 existente.

Entrada de consulta:

?canalPedido=TOTEM&status=PAGAMENTO_APROVADO

Saída esperada: Status 200. Todos os pedidos retornados possuem o canal TOTEM e o status PAGAMENTO_APROVADO.

Evidência: Requisição T04 - Filtrar pedidos por canal na pasta Pedidos.

Atualização completa do status

Tipo: Positivo

Endpoint e método: PATCH /pedidos/{id}/status

Pré-condição: Administrador autenticado e pedido no status PAGAMENTO_APROVADO.

Entradas executadas na ordem:

```
{  
  "status": "EM_PREPARO"  
}  
  
{  
  "status": "PRONTO"  
}  
  
{  
  "status": "ENTREGUE"  
}
```

Saída esperada: Cada requisição retorna 200. O pedido termina como ENTREGUE, o histórico registra as transições, a auditoria registra as atualizações e os pontos são creditados caso o cliente possua consentimento ativo.

Evidências:

- T05.1 - Pedido em preparo;
- T05.2 - Pedido pronto;
- T05.3 - Pedido entregue;
- imagem da tabela de auditoria ou do histórico no DBeaver.

Pagamento mock recusado

Tipo: Positivo

Endpoint e método: POST /pedidos

Pré-condição: Cliente autenticado e produto com estoque suficiente. O total deve ultrapassar R\$ 500,00, conforme a regra do mock.

Entrada:

```
{  
  "canalPedido": "WEB",  
  "clienteId": "00000000-0000-4000-8000-0000000000012",  
  "unidadeId": "00000000-0000-4000-8000-0000000000001",  
  "itens": [  
    {
```

```
"produtoId": "{{produtoPagamentoNegadId}}",
"quantidade": 30
}
],
"formaPagamento": "MOCK"
}
```

Saída esperada: Status 201, pois a simulação foi processada corretamente. O pagamento fica RECUSADO, o pedido fica CANCELADO e o estoque reservado é estornado.

Evidências:

- T06.1 - Criar pedido com pagamento recusado;
- T06.2 - Consultar pagamento recusado;
- registro ESTORNO no DBeaver.

Acesso sem token

Tipo: Negativo

Endpoint e método: GET /estoque

Pré-condição: A requisição deve ser enviada sem Bearer Token.

Entrada de consulta:

?unidadeId=00000000-0000-4000-8000-000000000001

Saída esperada: Status 401, código TOKEN_NAO_INFORMADO e mensagem padronizada.

Evidência: Requisição T07 - Estoque sem token na pasta Auth.

Cliente tentando cadastrar produto

Tipo: Negativo

Endpoint e método: POST /produtos

Pré-condição: Utilizar o token de um usuário com perfil CLIENTE.

Entrada:

```
{
  "nome": "Produto sem permissão",
  "descricao": "Teste de autorização",
```

```
"categoria": "LANCHE"
}
```

Saída esperada: Status 403 e código indicando que o perfil não possui permissão. Nenhum produto é cadastrado.

Evidência: Requisição T08 - Cliente tenta cadastrar produto na pasta Produtos.

Canal de pedido inválido

Tipo: Negativo

Endpoint e método: POST /pedidos

Pré-condição: Cliente autenticado e produto disponível.

Entrada:

```
{
  "canalPedido": "BALCAO",
  "clienteId": "00000000-0000-4000-8000-0000000000012",
  "unidadeId": "00000000-0000-4000-8000-0000000000001",
  "itens": [
    {
      "produtoId": "{{produtoId}}",
      "quantidade": 1
    }
  ],
  "formaPagamento": "MOCK"
}
```

Saída esperada: Status 422 e resposta padronizada de validação. O pedido não é criado porque os canais aceitos são somente APP, WEB e TOTEM.

Evidência: Requisição T09 - Canal de pedido inválido na pasta Pedidos.

Pedido com estoque insuficiente

Tipo: Negativo

Endpoint e método: POST /pedidos

Pré-condição: Cliente autenticado e produto existente no cardápio.

Entrada:

```
{
  "canalPedido": "APP",
  "clienteId": "00000000-0000-4000-8000-0000000000012",
  "unidadeId": "00000000-0000-4000-8000-0000000000001",
  "itens": [
    {
      "produtoId": "{{produtoId}}",
      "quantidade": 9999
    }
  ],
  "formaPagamento": "MOCK"
}
```

Saída esperada: Status 409, código ESTOQUE_INSUFICIENTE e nenhuma criação de pedido ou pagamento.

Evidência: Requisição T10 - Estoque insuficiente na pasta Pedidos.

21. Implementação, itens conceituais e evidências

A parte prática do projeto foi desenvolvida somente no back-end, utilizando Node.js, TypeScript, Express, PostgreSQL e Prisma ORM. A API possui autenticação por JWT, refresh token, logout, controle de acesso por perfis, unidades, produtos, cardápio, estoque, pedidos, pagamento mock, fidelidade, consentimento LGPD e auditoria.

Os pedidos aceitam os canais APP, WEB e TOTEM. Durante a criação, a API verifica o cardápio e a quantidade disponível no estoque da unidade. Depois disso, o estoque é reservado e o pagamento mock é processado. Quando aprovado, o pedido pode avançar até a entrega. Quando recusado, o pedido é cancelado e o estoque é estornado.

A persistência foi feita em PostgreSQL, com migrations, seed e Prisma ORM. Os endpoints foram documentados pelo Swagger/OpenAPI e os testes foram organizados em uma coleção Postman.

Itens mantidos de forma conceitual

O front-end dos canais APP, WEB e TOTEM não foi desenvolvido, pois a entrega está concentrada na solução back-end. Esses canais foram representados pelo campo canalPedido e pelos contratos da API.

As promoções e campanhas foram apresentadas como regras e propostas de funcionamento, sem a implementação de um motor automático de descontos. O pagamento também não utiliza um provedor real, seguindo a exigência de representar apenas uma integração simulada.

As estratégias avançadas de desempenho, como balanceamento de carga, cache distribuído e escalabilidade horizontal, foram mantidas como propostas. Na implementação atual foram utilizadas paginação, consultas filtradas, índices no banco e transações para manter a consistência.

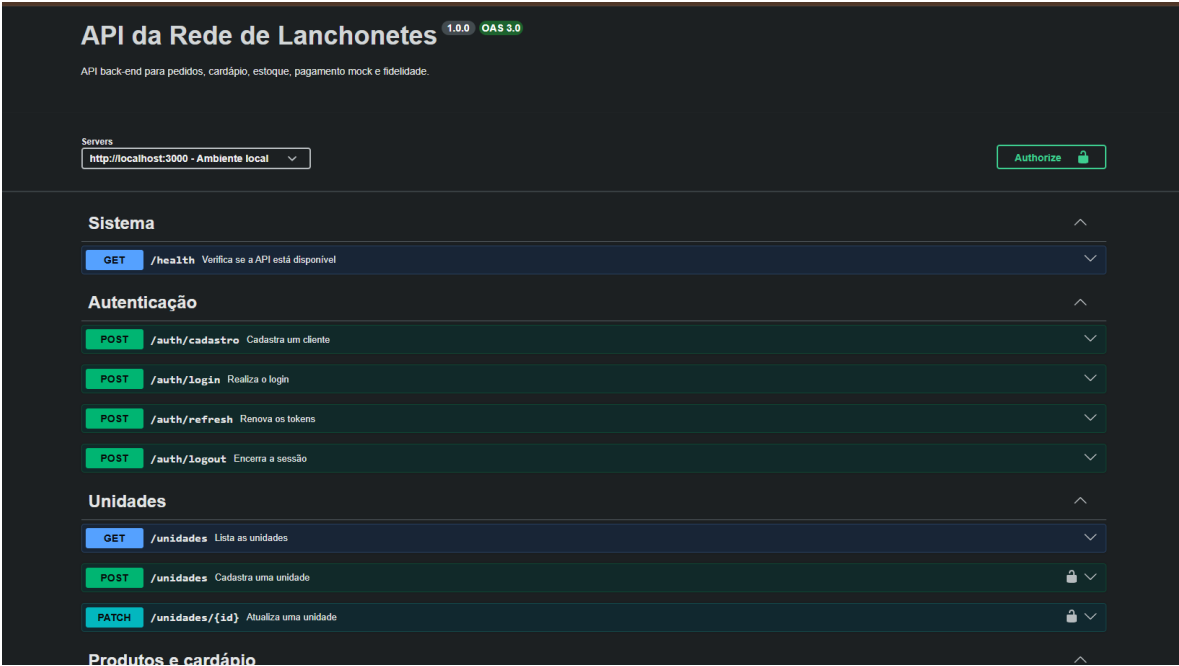
Essas escolhas mantiveram o projeto pequeno e deram prioridade ao fluxo obrigatório de pedidos, estoque, pagamento, atualização de status, segurança e fidelidade.

Evidências de funcionamento

As evidências demonstram que a API foi executada com banco de dados real e que os principais recursos podem ser testados novamente. O Swagger apresenta os endpoints, parâmetros, autenticação, exemplos e códigos HTTP. A coleção Postman contém os seis cenários positivos e quatro negativos exigidos.

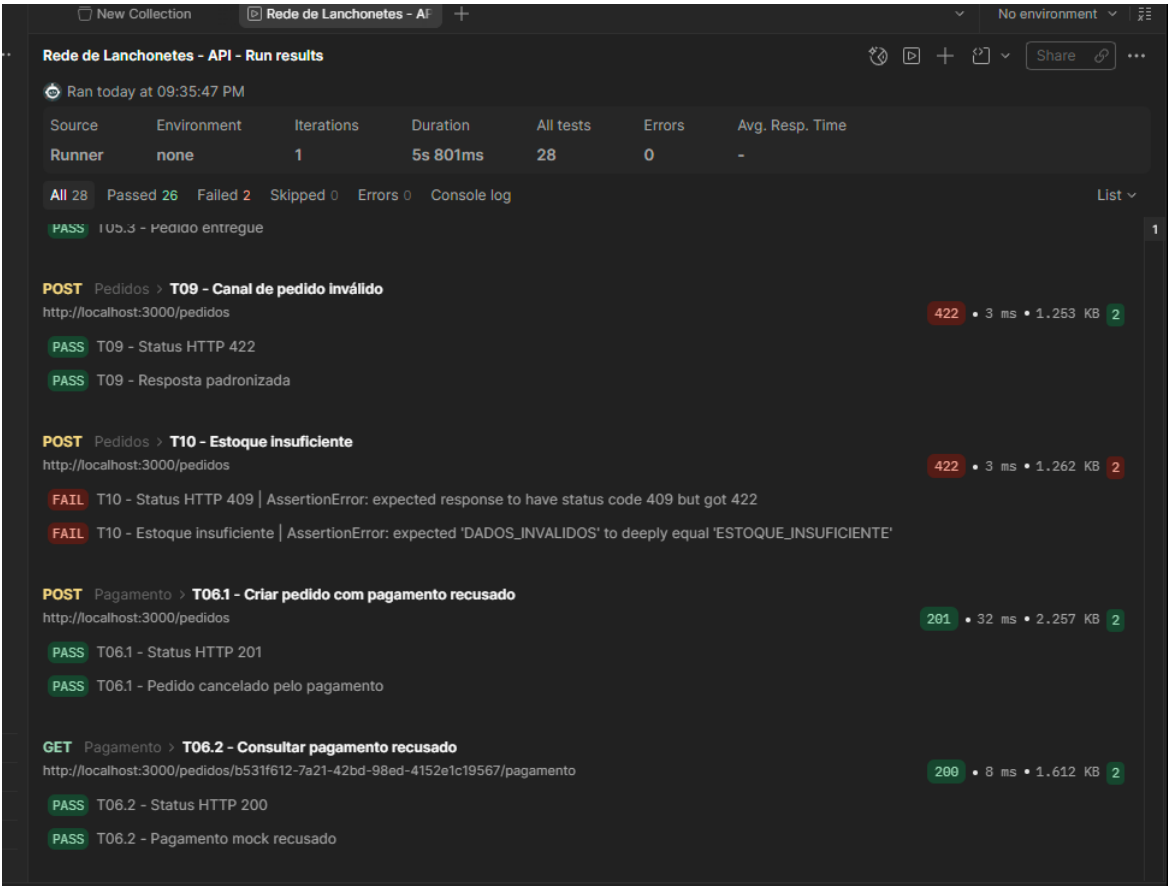
O banco de dados também registra os pedidos, pagamentos, movimentações de estoque, histórico de status, pontos de fidelidade e logs de auditoria. Dessa forma, é possível verificar a conexão entre as requisições da API e os registros persistidos.

Figura 1 – Documentação dos endpoints no Swagger



Fonte: Autoria própria (2026).

Figura 2 – Execução dos testes no Postman



Fonte: Autoria própria (2026).

Figura 3 – Registros de pedidos e pagamentos no PostgreSQL

	id	pedido_id	AZ provedor	AZ status	123 valor	AZ transacao_externa
1	3eacff3-09d0-490c-8bb4-783ce5c406a7	51bc00ec-55df-4d24-868b-4493f7c2d7a7	MOCK	APROVADO	12	MOCK-10569b04-cf94-
2	09842597-2b42-4efc-903d-d544d648ef60	4c096450-5506-4d71-9382-d8332da669c1	MOCK	RECUSADO	567	MOCK-92978699-014b-
3	393676f3-0ba0-422c-aa4d-f2ebb1ea23ab	26d275d-9296-47ed-bec4-11495eb79771	MOCK	APROVADO	10	MOCK-86b7efff-45d3-4
4	d3c6a99c-4226-494e-8c88-fc3d2d7f7f64	2a1c9acc-4b77-44eb-8fa6-e881b094c1b4	MOCK	APROVADO	10	MOCK-6291496e-adt3-
5	1797a115-bde1-4891-907c-065arf2c53b2	c93aa44-7eb8-4a13-9236-371fa99a997	MOCK	APROVADO	10	MOCK-e8b5c10f-c2b5-
6	0fbda29e-c8cf-48b6-938b-340fd2ba0e05	b331f812-7a21-42bd-88ed-4152e1c19567	MOCK	RECUSADO	567	MOCK-c3bb5d53-22d6-

Fonte: Autoria própria (2026).

Figura 4 – Logs de auditoria das ações realizadas

Propriedades

Dados

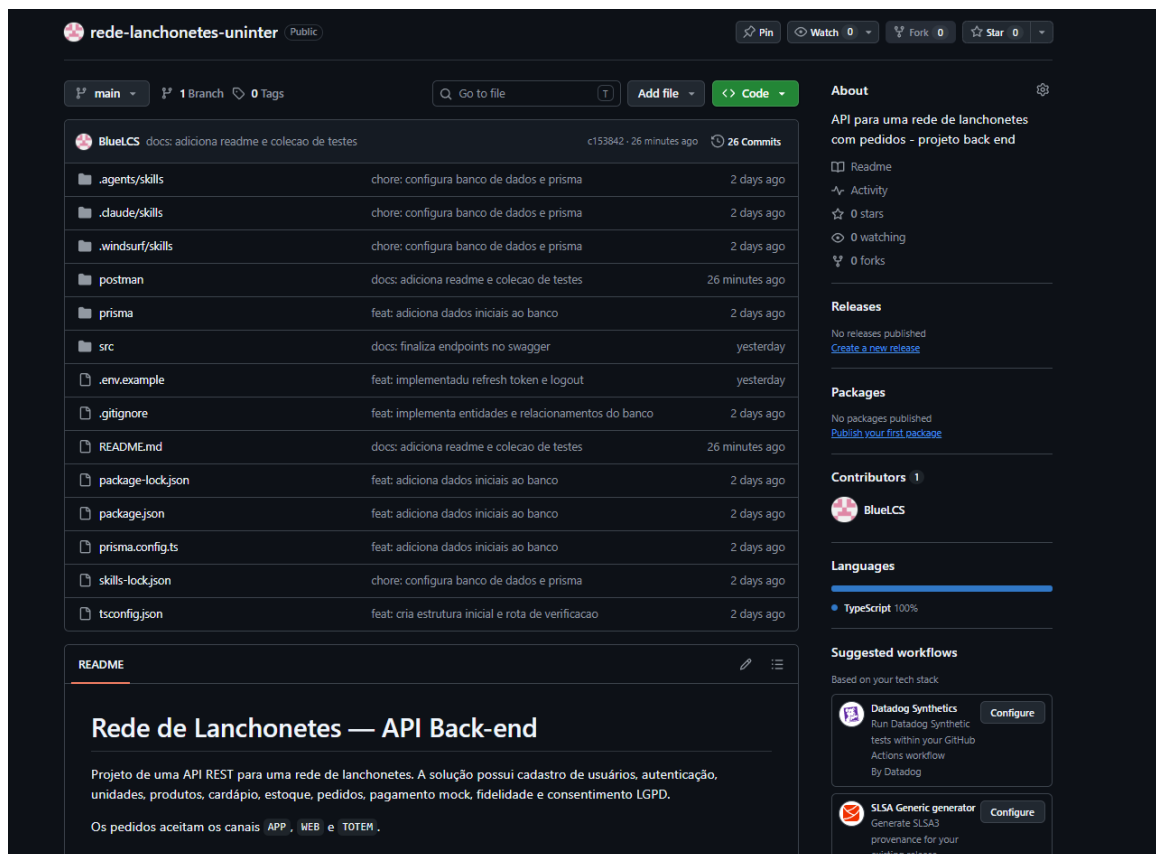
Diagrama

SQL

Insira uma expressão SQL para filtrar os resultados (use Ctrl+Espaço)

Fonte: Autoria própria (2026).

Figura 5 – Código e histórico de commits no GitHub



Fonte: Autoria própria (2026).

22. Repositório do projeto

O código-fonte, as migrations, o seed, a documentação Swagger/OpenAPI, a coleção Postman, o README e os arquivos dos diagramas estão disponíveis no repositório:

<https://github.com/BlueLCS/rede-lanchonetes-uninter>

O README apresenta os requisitos, a configuração das variáveis de ambiente, a preparação do banco, os comandos de execução e a ordem dos testes. Entre os arquivos de modelagem disponíveis estão DER.png, DER.svg, Diagrama de caso de uso.png e diagrama de classe.png.

23. Conclusão

O projeto entregou uma API back-end para uma rede de lanchonetes, com suporte aos canais app web e totem. A prioridade foi implementar um fluxo completo e testável, começando pela autenticação e seguindo pela consulta do cardápio, controle de estoque, criação do pedido, pagamento mock, atualização de status e fidelidade.

O DER definiu as entidades e os relacionamentos utilizados no PostgreSQL. O diagrama de classes representou os principais objetos do domínio, enquanto os casos de uso mostraram as ações dos clientes, atendentes, cozinha, gerentes e gateway de pagamento. Esses modelos se conectam aos endpoints e ao fluxo crítico implementado.

O pagamento externo foi representado por um serviço mock. Pedidos de até R\$ 500,00 são aprovados e pedidos acima desse valor são recusados. Quando ocorre uma recusa, o pedido é cancelado e o estoque reservado é devolvido. A API também trata estoque insuficiente, canais inválidos, transições incorretas e acesso sem permissão por meio de respostas padronizadas.

Na parte de segurança foram utilizados hash de senha, JWT, refresh token, autorização por perfis e auditoria. O consentimento LGPD registra a finalidade, a versão do termo, a escolha do cliente, a data e o IP. Dados como senha, hash e tokens armazenados não são expostos nas respostas.

Os testes positivos e negativos demonstram o funcionamento da autenticação, autorização, validação, pedidos, estoque, pagamento e atualização de status. A coleção Postman, o Swagger e os registros do banco permitem que o avaliador repita o fluxo.

Referências

BRASIL. Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais (LGPD). Brasília, DF: Presidência da República, 2018. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm. Acesso em: 11 ago. 2026.

EXPRESS.JS. Express documentation. Disponível em: <https://expressjs.com/>. Acesso em: 11 ago. 2026.

NODE.JS. Node.js documentation. Disponível em: <https://nodejs.org/docs/latest/api/>. Acesso em: 11 ago. 2026.

OPENAPI INITIATIVE. OpenAPI Specification. Disponível em: <https://spec.openapis.org/oas/latest.html>. Acesso em: 11 ago. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. PostgreSQL documentation. Disponível em: <https://www.postgresql.org/docs/>. Acesso em: 11 ago. 2026.

POSTMAN. Postman Learning Center. Disponível em: <https://learning.postman.com/docs/>. Acesso em: 11 ago. 2026.

PRISMA. Prisma ORM documentation. Disponível em: <https://www.prisma.io/docs>. Acesso em: 11 ago. 2026.